```
# ===== backend/app/services/agents/persistent_repository.py =====
"""SqlAlchemy 持久化仓储：内存默认、数据库可选。
实现现有 `RunRepository` / `ApprovalRepository` 协议（repositories.py），
不修改协议语义；内存仓储保持默认、数据库不可用时离线测试不受影响。
使用同步 SQLAlchemy Engine + 独立 Session（供已有的同步 Runtime 调用）：
  - Engine 与 Session 工厂可注入：测试注入 SQLite 内存（StaticPool 共享
    单连接，保证「跨仓储实例」可见），生产注入 PostgreSQL 同步驱动。
  - 所有写操作在单事务内完成；冲突回滚并保留原数据（不静默覆盖）。
  - 幂等键唯一部分索引（t_agent_run_state.idempotency_key）保证：
    相同幂等键并发创建时最多产生一个 run。
  - state_version 乐观并发：保存 run 时库内版本与调用方观察版本不一致，
    抛现有 `TaskConflictError`（不静默覆盖）；成功保存 +1。
  - 步骤写入保持 (run_id, step_no) 唯一约束，不覆盖历史步骤。
  - 审批创建/查询/决策全部落 t_agent_approval，跨进程/跨仓储实例可见，
    已决策的审批不允许被覆盖（cancelled 允许被正式决策覆盖，同内存语义）。
  - close() 释放连接池，避免测试泄漏连接。
续跑状态持久化的关键设计（runtime.py 为冻结文件，不得修改）：
  AgentRuntime 只在终态调用 save()；中间状态（stage / status / 审批挂起）
  的内存变更无法被 save() 捕获。因此本仓储在每次仓储写入时对已知的
  run 对象做一次「检查点快照」：把当前 status + request + runtime_state
  同步到 t_agent_run / t_agent_run_state（不递增 state_version，last-write-wins
  崩溃恢复语义），并在 `steps()` 读取路径上同样刷新：这样 `run()` 返回
  前 `_result()` 的 `steps()` 调用能捕获「waiting_approval」等 stage 翻转，
  重启后 resume() 从数据库精确续跑，不重放规划/策略/审批阶段。
解密预算：request_json / runtime_state_json 写入前统一 `_sanitize`：
  丢弃敏感键（api_key / authorization / chain_of_thought / reasoning /
  password / secret / credential / *_token 等，键名也不得进入持久化内容），
  字符串值中的敏感词替换为 [REDACTED]，长值截断为摘要长度。
 """
from __future__ import annotations
import json
import logging
import re
from datetime import datetime, timezone
from typing import Any
from sqlalchemy import Engine, func, select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session, sessionmaker
from app.models.agent import (
    AgentApproval as AgentApprovalORM,
    AgentRun as AgentRunORM,
    AgentStep as AgentStepORM,
  )
from app.models.agent_state import AgentRunState as AgentRunStateORM
from app.services.agents.errors import AgentNotFoundError, TaskConflictError
from app.services.agents.model import AgentRun, AgentRunRequest, AgentStep, Expectation, RuleStep
from app.services.agents.repositories import ApprovalRecord
logger = logging.getLogger("oceanus.agents.persistent")
# ---------------------------------------------------------------------------
# 脱敏（解密预算）—— 键名命中即整体丢弃，字符串值中的敏感词替换
```

```
# ----------------------------------------------------------------------



_FORBIDDEN_KEY_RE = re.compile(
    r"(?i)(api[_-]?key|authorization|chain[_-]?of[_-]?thought|reasoning|"
    r"private[_-]?reasoning|raw[_-]?reasoning|full[_-]?input|full[_-]?output|"
    r"password|passwd|secret|credential|access[_-]?token|auth[_-]?token)"
  )
# 敏感字段探测：测试 8 断言持久化内容不得出现这些字面量
SENSITIVE_TERMS: tuple[str, ...] = (
    "api_key",
    "authorization",
    "chain_of_thought",
    "reasoning",
  )
_MAX_STRING_LEN = 500  # 摘要长度上限：只存续跑所需业务摘要
def _sanitize(value: Any, *, depth: int = 0) -> Any:
    """递归脱敏：敏感键丢弃、敏感词替换、长字符串截断。"""
    if depth > 8:
        return "..."
    if isinstance(value, dict):
        out: dict[str, Any] = {}
        for key, item in value.items():
            if isinstance(key, str) and _FORBIDDEN_KEY_RE.search(key):
                continue  # 敏感键整体丢弃（键名也不得进入持久化内容）
            out[key] = _sanitize(item, depth=depth + 1)
        return out
    if isinstance(value, (list, tuple)):
        return [_sanitize(item, depth=depth + 1) for item in value]
    if isinstance(value, str):
        cleaned = _FORBIDDEN_KEY_RE.sub("[REDACTED]", value)
        return cleaned if len(cleaned) <= _MAX_STRING_LEN else cleaned[:_MAX_STRING_LEN] + "..."
    return value
def _json_default(value: Any) -> str:
    """json.dumps 兜底：datetime / Decimal / set 等转为可序列化形式。"""
    if isinstance(value, datetime):
        return value.isoformat()
    if isinstance(value, (set, frozenset)):
        return sorted(str(v) for v in value)
    if hasattr(value, "isoformat"):
        return str(value.isoformat())
    return str(value)
# ----------------------------------------------------------------------
# AgentRunRequest <-> JSON（脱敏后）
# ----------------------------------------------------------------------
def _serialize_request(request: AgentRunRequest | None) -> str:
    if request is None:
        return "{}"
    payload = {
        "trigger_type": request.trigger_type,
        "objective": request.objective,
         "actor": request.actor,
```

```
        "role": request.role,
        "params": request.params or {},
        "idempotency_key": request.idempotency_key,
        "trace_id": request.trace_id,
        "plan_key": request.plan_key,
        # 经验检索上下文：续跑/回放时要能还原"当时内核知道哪些业务事实"，
        # 否则同一条 run 重放会给出不同的经验引用。
        "lessons_context": request.lessons_context or {},
    }
    return json.dumps(_sanitize(payload), ensure_ascii=False, sort_keys=True, default=_json_default)
def _deserialize_request(raw: str | None) -> AgentRunRequest | None:
    if not raw:
        return None
    try:
        data = json.loads(raw)
    except (ValueError, TypeError):
        return None
    if not isinstance(data, dict):
        return None
    return AgentRunRequest(
        trigger_type=str(data.get("trigger_type") or "event"),
        objective=str(data.get("objective") or ""),
        actor=str(data.get("actor") or "operator"),
        role=str(data.get("role") or "operator"),
        params=dict(data.get("params") or {}),
        idempotency_key=data.get("idempotency_key"),
        trace_id=data.get("trace_id"),
        plan_key=data.get("plan_key"),
        lessons_context=dict(data.get("lessons_context") or {}),
    )
# ---------------------------------------------------------------------
# runtime_state <-> JSON（只保留续跑所需字段，脱敏后）
# ---------------------------------------------------------------------
# 续跑/回放字段：阶段、计划游标、重规划计数、挂起审批 ID、业务任务结果、
# 步骤角色归属（step_no → 角色）。step_roles 不参与续跑判断，但必须跟着
# 状态快照一起落库，否则服务重启后「只读回放」的轨迹会丢掉角色徽标。
_CONTINUATION_KEYS: tuple[str, ...] = (
    "stage",
    "plan_idx",
    "replan_count",
    "approvals",
    "task_result",
    "step_roles",
)
def _expectation_to_dict(expect: Expectation) -> dict[str, Any]:
    return {
        "key": expect.key,
        "min_items": expect.min_items,
        "contains": list(expect.contains),
        "on_violation": expect.on_violation,
```

```
        "error_code": expect.error_code,
    }
def _expectation_from_dict(data: dict[str, Any]) -> Expectation:
    return Expectation(
        key=str(data.get("key") or ""),
        min_items=data.get("min_items"),
        contains=tuple(data.get("contains") or ()),
        on_violation=str(data.get("on_violation") or "terminate"),
        error_code=str(data.get("error_code") or "tool_failed"),
    )
def _rule_step_to_dict(step: RuleStep) -> dict[str, Any]:
    return {
        "tool": step.tool,
        "input": step.input,
        "summary": step.summary,
        "expect": _expectation_to_dict(step.expect) if step.expect is not None else None,
        "on_error": step.on_error,
        "role": step.role,
    }
def _rule_step_from_dict(data: dict[str, Any]) -> RuleStep:
    expect_raw = data.get("expect")
    expect = None
    if isinstance(expect_raw, dict):
        expect = _expectation_from_dict(expect_raw)
    role = data.get("role")
    return RuleStep(
        tool=str(data.get("tool") or ""),
        input=dict(data.get("input") or {}),
        summary=str(data.get("summary") or ""),
        expect=expect,
        on_error=str(data.get("on_error") or "terminate"),
        role=str(role) if role else None,
    )
def _serialize_runtime_state(state: dict[str, Any]) -> str:
    payload: dict[str, Any] = {}
    for key in _CONTINUATION_KEYS:
        if key in state:
            payload[key] = state[key]
    plan = state.get("plan")
    if isinstance(plan, list):
        payload["plan"] = [_rule_step_to_dict(s) for s in plan if isinstance(s, RuleStep)]
    bindings = state.get("bindings")
    if isinstance(bindings, dict):
        payload["bindings"] = bindings
    return json.dumps(_sanitize(payload), ensure_ascii=False, sort_keys=True, default=_json_default)
def _deserialize_runtime_state(raw: str | None) -> dict[str, Any]:
    if not raw:
        return {}
    try:
        data = json.loads(raw)
```

```
    except (ValueError, TypeError):
        return {}
    if not isinstance(data, dict):
        return {}
    state: dict[str, Any] = {}
    for key in _CONTINUATION_KEYS:
        if key in data:
            state[key] = data[key]
    plan = data.get("plan")
    if isinstance(plan, list):
        state["plan"] = [_rule_step_from_dict(p) for p in plan if isinstance(p, dict)]
    bindings = data.get("bindings")
    if isinstance(bindings, dict):
        state["bindings"] = bindings
    # 兼容：老数据可能缺默认游标字段
    if "plan" in state and "plan_idx" not in state:
        state["plan_idx"] = 0
    if "replan_count" not in state:
        state["replan_count"] = 0
    if "stage" not in state:
        state["stage"] = "created"
    return state
# ---------------------------------------------------------------------
# ORM 行 <-> 领域对象
# ---------------------------------------------------------------------
def _row_to_run(run_row: AgentRunORM, state_row: AgentRunStateORM | None) -> AgentRun:
    return AgentRun(
        run_id=run_row.run_id,
        trigger_type=run_row.trigger_type,
        objective=run_row.objective,
        status=run_row.status,
        request=_deserialize_request(state_row.request_json) if state_row is not None else None,
        policy_version=run_row.policy_version,
        started_at=run_row.started_at,
        finished_at=run_row.finished_at,
        termination_reason=run_row.termination_reason,
        trace_id=run_row.trace_id,
        created_at=run_row.created_at,
        updated_at=run_row.updated_at,
        runtime_state=_deserialize_runtime_state(state_row.runtime_state_json)
        if state_row is not None
        else {},
    )
def _apply_run_row(row: AgentRunORM, run: AgentRun) -> None:
    """把领域 run 的当前状态写回 t_agent_run 行（create/save/快照共用）。"""
    row.trigger_type = run.trigger_type
    row.objective = run.objective
    row.status = run.status
    row.policy_version = run.policy_version
    row.started_at = run.started_at
```

```
    row.finished_at = run.finished_at



    row.termination_reason = run.termination_reason
    row.trace_id = run.trace_id
    row.created_at = run.created_at
    row.updated_at = run.updated_at
def _row_to_step(row: AgentStepORM) -> AgentStep:
    return AgentStep(
        step_id=row.step_id,
        run_id=row.run_id,
        step_no=row.step_no,
        step_type=row.step_type,
        decision_summary=row.decision_summary,
        tool_name=row.tool_name,
        tool_version=row.tool_version,
        input_hash=row.input_hash,
        output_hash=row.output_hash,
        status=row.status,
        latency_ms=row.latency_ms,
        error_code=row.error_code,
        created_at=row.created_at,
    )
def _row_to_approval(row: AgentApprovalORM) -> ApprovalRecord:
    return ApprovalRecord(
        approval_id=row.approval_id,
        run_id=row.run_id,
        requested_action=row.requested_action,
        risk_level=row.risk_level,
        requested_by=row.requested_by,
        decided_by=row.decided_by,
        decision=row.decision,
        reason=row.reason,
        requested_at=row.requested_at,
        decided_at=row.decided_at,
    )
# ---------------------------------------------------------------------
# Run 仓储（实现 RunRepository 协议）
# ---------------------------------------------------------------------
class SqlAlchemyRunRepository:
    """同步 SQLAlchemy 运行仓储（协议：repositories.RunRepository）。
    Engine 与 Session 工厂可注入；所有写操作在事务内完成，冲突回滚并
    保留原数据。`close()` 释放连接池。
    """
    def __init__(
        self,
        engine: Engine,
        session_factory: sessionmaker[Session] | None = None,
    ) -> None:
        self._engine = engine
        self._factory = session_factory or sessionmaker(bind=engine, expire_on_commit=False)
        # 本实例观察到的 state_version（乐观并发检查的依据）
```

```
        self._versions: dict[str, int] = {}



        # 本实例接触过的 run 对象（用于中间状态检查点快照）
        self._live: dict[str, AgentRun] = {}
    # ------------------------------------------------------------------
    # 资源释放
    # ------------------------------------------------------------------
    def close(self) -> None:
        """释放连接池（测试防泄漏；重复调用安全）。"""
        self._engine.dispose()
    def _session(self) -> Session:
        return self._factory()
    # ------------------------------------------------------------------
    # 检查点快照：把 run 的当前内存状态同步到库（不递增 state_version）
    # ------------------------------------------------------------------
    def _flush_snapshot(self, run: AgentRun | None) -> None:
        """把 run 当前 status + request + runtime_state 同步到库（崩溃恢复用）。
        不参与乐观并发：last-write-wins；失败只告警不抛出（读取路径触发）。
        """
        if run is None:
            return
        try:
            with self._session() as session:
                run_row = session.execute(
                    select(AgentRunORM).where(AgentRunORM.run_id == run.run_id)
                ).scalar_one_or_none()
                state_row = session.execute(
                    select(AgentRunStateORM).where(AgentRunStateORM.run_id == run.run_id)
                ).scalar_one_or_none()
                if run_row is None or state_row is None:
                    return
                _apply_run_row(run_row, run)
                state_row.request_json = _serialize_request(run.request)
                state_row.runtime_state_json = _serialize_runtime_state(run.runtime_state)
                session.commit()
        except Exception:  # noqa: BLE001：快照失败不影响主流程
            logger.warning("[AgentPersist] run %s 状态快照写入失败（忽略）", run.run_id, exc_info=T…
    # ------------------------------------------------------------------
    # RunRepository 协议方法
    # ------------------------------------------------------------------
    def create(self, run: AgentRun) -> None:
        if not isinstance(run, AgentRun):
            raise TypeError(f"run 必须是 AgentRun：{type(run).__name__}")
        with self._session() as session:
            try:
                session.add(
                    AgentRunORM(
                        run_id=run.run_id,
                        trigger_type=run.trigger_type,
                        objective=run.objective,
                        status=run.status,
```

```
                    policy_version=run.policy_version,
                    started_at=run.started_at,
                    finished_at=run.finished_at,
                    termination_reason=run.termination_reason,
                    trace_id=run.trace_id,
                    created_at=run.created_at,
                    updated_at=run.updated_at,
                )
            )
            session.add(
                AgentRunStateORM(
                    run_id=run.run_id,
                    request_json=_serialize_request(run.request),
                    runtime_state_json=_serialize_runtime_state(run.runtime_state),
                    idempotency_key=run.request.idempotency_key if run.request is not None else…
                    state_version=1,
                )
            )
            session.commit()
        except IntegrityError as exc:
            session.rollback()
            self._raise_create_conflict(run, exc)
        else:
            self._versions[run.run_id] = 1
            self._live[run.run_id] = run
def save(self, run: AgentRun) -> None:
    if not isinstance(run, AgentRun):
        raise TypeError(f"run 必须是 AgentRun：{type(run).__name__}")
    with self._session() as session:
        state_row = session.execute(
            select(AgentRunStateORM).where(AgentRunStateORM.run_id == run.run_id)
        ).scalar_one_or_none()
        if state_row is None:
            session.rollback()
            raise AgentNotFoundError(f"run 不存在：{run.run_id}")
        expected = self._versions.get(run.run_id)
        if expected is None:
            # 本实例从未观察过该 run（如新建实例直接保存）→ 以库内当前值为准
            expected = state_row.state_version
        if state_row.state_version != expected:
            session.rollback()
            raise TaskConflictError(
                f"run {run.run_id} 状态版本冲突：期望 v{expected}，库内 v{state_row.state_versi…
            )
        new_version = state_row.state_version + 1
        state_row.request_json = _serialize_request(run.request)
        state_row.runtime_state_json = _serialize_runtime_state(run.runtime_state)
        state_row.state_version = new_version
        run_row = session.execute(
            select(AgentRunORM).where(AgentRunORM.run_id == run.run_id)
```

```
        ).scalar_one_or_none()


        if run_row is None:
            session.rollback()
            raise AgentNotFoundError(f"run 不存在：{run.run_id}")
        _apply_run_row(run_row, run)
        try:
            session.commit()
        except IntegrityError as exc:
            session.rollback()
            raise TaskConflictError(f"run {run.run_id} 保存冲突") from exc
        else:
            self._versions[run.run_id] = new_version
            self._live[run.run_id] = run
def get(self, run_id: str) -> AgentRun | None:
    if not isinstance(run_id, str):
        raise TypeError(f"run_id 必须是 str：{type(run_id).__name__}")
    # 读取路径顺带刷新本实例已知的检查点快照（见模块 docstring）
    self._flush_snapshot(self._live.get(run_id))
    with self._session() as session:
        run_row = session.execute(
            select(AgentRunORM).where(AgentRunORM.run_id == run_id)
        ).scalar_one_or_none()
        if run_row is None:
            return None
        state_row = session.execute(
            select(AgentRunStateORM).where(AgentRunStateORM.run_id == run_id)
        ).scalar_one_or_none()
        if state_row is not None:
            self._versions[run_id] = state_row.state_version
        run = _row_to_run(run_row, state_row)
        self._live[run_id] = run
        return run
def list(
    self,
    *,
    status: str | None = None,
    limit: int | None = None,
) -> list[AgentRun]:
    if status is not None and not isinstance(status, str):
        raise TypeError(f"status 必须是 str 或 None：{type(status).__name__}")
    if limit is not None and (not isinstance(limit, int) or isinstance(limit, bool) or limit < …
        raise TypeError(f"limit 必须是非负整数或 None：{limit!r}")
    with self._session() as session:
        stmt = (
            select(AgentRunORM, AgentRunStateORM)
            .outerjoin(AgentRunStateORM, AgentRunStateORM.run_id == AgentRunORM.run_id)
            .order_by(AgentRunORM.created_at)
        )
        if status is not None:
            stmt = stmt.where(AgentRunORM.status == status)
```

```
        if limit is not None:
            stmt = stmt.limit(limit)
        runs: list[AgentRun] = []
        for run_row, state_row in session.execute(stmt).all():
            if state_row is not None:
                self._versions[run_row.run_id] = state_row.state_version
            run = _row_to_run(run_row, state_row)
            self._live[run_row.run_id] = run
            runs.append(run)
        return runs
def find_by_idempotency_key(self, key: str) -> AgentRun | None:
    if not isinstance(key, str) or not key:
        raise TypeError(f"key 必须是非空 str：{key!r}")
    with self._session() as session:
        state_row = session.execute(
            select(AgentRunStateORM).where(AgentRunStateORM.idempotency_key == key)
        ).scalar_one_or_none()
        if state_row is None:
            return None
        run_row = session.execute(
            select(AgentRunORM).where(AgentRunORM.run_id == state_row.run_id)
        ).scalar_one_or_none()
        if run_row is None:
            return None
        self._versions[run_row.run_id] = state_row.state_version
        run = _row_to_run(run_row, state_row)
        self._live[run_row.run_id] = run
        return run
def add_step(self, step: AgentStep) -> None:
    if not isinstance(step, AgentStep):
        raise TypeError(f"step 必须是 AgentStep：{type(step).__name__}")
    # 步骤写入前刷新该 run 的检查点快照（捕获 stage 翻转，见模块 docstring）
    self._flush_snapshot(self._live.get(step.run_id))
    with self._session() as session:
        session.add(
            AgentStepORM(
                step_id=step.step_id,
                run_id=step.run_id,
                step_no=step.step_no,
                step_type=step.step_type,
                decision_summary=step.decision_summary,
                tool_name=step.tool_name,
                tool_version=step.tool_version,
                input_hash=step.input_hash,
                output_hash=step.output_hash,
                status=step.status,
                latency_ms=step.latency_ms,
                error_code=step.error_code,
                created_at=step.created_at,
            )
```

```
            )



            try:
                session.commit()
            except IntegrityError as exc:
                session.rollback()
                raise TaskConflictError(
                    f"步骤写入冲突 ((run_id, step_no) 或 step_id 重复)：
                    f"{step.run_id} #{step.step_no} ({step.step_id}) "
                ) from exc
    def steps(self, run_id: str) -> list[AgentStep]:
        if not isinstance(run_id, str):
            raise TypeError(f"run_id 必须是 str：{type(run_id).__name__}")
        # 读取路径顺带刷新本实例已知的检查点快照（捕获终态前的 stage 翻转）
        self._flush_snapshot(self._live.get(run_id))
        with self._session() as session:
            rows = session.execute(
                select(AgentStepORM)
                .where(AgentStepORM.run_id == run_id)
                .order_by(AgentStepORM.step_no)
            ).scalars().all()
            return [_row_to_step(row) for row in rows]
    def count(self) -> int:
        with self._session() as session:
            return int(session.execute(select(func.count()).select_from(AgentRunORM)).scalar_one())
    # ------------------------------------------------------------------
    # 内部工具
    # ------------------------------------------------------------------
    def _raise_create_conflict(self, run: AgentRun, exc: IntegrityError) -> None:
        """区分 run_id 重复与幂等键重复，统一抛 TaskConflictError。"""
        key = run.request.idempotency_key if run.request is not None else None
        if key:
            existing = self.find_by_idempotency_key(key)
            if existing is not None and existing.run_id != run.run_id:
                raise TaskConflictError(f"幂等键重复：{key}（已绑定 run {existing.run_id}）") from …
        raise TaskConflictError(f"run_id 重复：{run.run_id}") from exc
# ----------------------------------------------------------------------
# 审批仓储（实现 ApprovalRepository 协议）
# ----------------------------------------------------------------------
class SqlAlchemyApprovalRepository:
    """同步 SQLAlchemy 审批仓储（协议：repositories.ApprovalRepository）。
    审批创建/查询/决策持久化到 t_agent_approval，跨进程/跨仓储实例可见；
    已决策的审批不允许被覆盖（cancelled 允许被正式决策覆盖，同内存语义）。
    """
    def __init__(
        self,
        engine: Engine,
        session_factory: sessionmaker[Session] | None = None,
    ) -> None:
        self._engine = engine
        self._factory = session_factory or sessionmaker(bind=engine, expire_on_commit=False)
```

```
def close(self) -> None:
    """释放连接池（测试防泄漏；重复调用安全）。"""
    self._engine.dispose()
def _session(self) -> Session:
    return self._factory()
def create(self, record: ApprovalRecord) -> ApprovalRecord:
    if not isinstance(record, ApprovalRecord):
        raise TypeError(f"record 必须是 ApprovalRecord：{type(record).__name__}")
    with self._session() as session:
        session.add(
            AgentApprovalORM(
                approval_id=record.approval_id,
                run_id=record.run_id,
                requested_action=record.requested_action,
                risk_level=record.risk_level,
                requested_by=record.requested_by,
                decided_by=record.decided_by,
                decision=record.decision,
                reason=record.reason,
                requested_at=record.requested_at,
                decided_at=record.decided_at,
            )
        )
        try:
            session.commit()
        except IntegrityError as exc:
            session.rollback()
            raise TaskConflictError(f"approval_id 重复：{record.approval_id}") from exc
    return record
def get(self, approval_id: str) -> ApprovalRecord | None:
    if not isinstance(approval_id, str):
        raise TypeError(f"approval_id 必须是 str：{type(approval_id).__name__}")
    with self._session() as session:
        row = session.execute(
            select(AgentApprovalORM).where(AgentApprovalORM.approval_id == approval_id)
        ).scalar_one_or_none()
        return _row_to_approval(row) if row is not None else None
def list_pending(self, run_id: str | None = None) -> list[ApprovalRecord]:
    if run_id is not None and not isinstance(run_id, str):
        raise TypeError(f"run_id 必须是 str 或 None：{type(run_id).__name__}")
    with self._session() as session:
        stmt = (
            select(AgentApprovalORM)
            .where(AgentApprovalORM.decision.is_(None))
            .order_by(AgentApprovalORM.requested_at)
        )
        if run_id is not None:
            stmt = stmt.where(AgentApprovalORM.run_id == run_id)
        rows = session.execute(stmt).scalars().all()
        return [_row_to_approval(row) for row in rows]
```

```
    def list_all(self) -> list[ApprovalRecord]:
        with self._session() as session:
            rows = session.execute(
                select(AgentApprovalORM).order_by(AgentApprovalORM.requested_at)
            ).scalars().all()
            return [_row_to_approval(row) for row in rows]
    def decide(
        self,
        approval_id: str,
        decided_by: str,
        decision: str,
        reason: str | None = None,
    ) -> ApprovalRecord:
        if not isinstance(approval_id, str):
            raise TypeError(f"approval_id 必须是 str：{type(approval_id).__name__}")
        if not isinstance(decided_by, str):
            raise TypeError(f"decided_by 必须是 str：{type(decided_by).__name__}")
        if decision not in ("approved", "rejected", "cancelled"):
            raise ValueError(f"非法决策：{decision!r}")
        if reason is not None and not isinstance(reason, str):
            raise TypeError(f"reason 必须是 str 或 None：{type(reason).__name__}")
        with self._session() as session:
            row = session.execute(
                select(AgentApprovalORM).where(AgentApprovalORM.approval_id == approval_id)
            ).scalar_one_or_none()
            if row is None:
                session.rollback()
                raise AgentNotFoundError(f"审批不存在：{approval_id}")
            if row.decision is not None and row.decision != "cancelled":
                session.rollback()
                raise TaskConflictError(f"审批 {approval_id} 已决策：{row.decision}")
            if decision not in ("approved", "rejected", "cancelled"):
                session.rollback()
                raise AgentNotFoundError(f"非法审批决策：{decision}")
            row.decision = decision
            row.decided_by = decided_by
            row.reason = reason
            row.decided_at = datetime.now(timezone.utc)
            session.commit()
            return _row_to_approval(row)
    def count(self) -> int:
        with self._session() as session:
            return int(
                session.execute(select(func.count()).select_from(AgentApprovalORM)).scalar_one()
            )
# ===== backend/app/services/agents/model_adapter.py =====
"""WP-11 可选大模型适配层与规则兜底（backend/app/services/agents/model_adapter.py）。
证据等级：E1/E2：本模块只提供「模型作为可选规划增强」的适配与校验能力，
不绑定任何厂商、不访问公网、不宣称真实部署。任何真实网络实现都必须默认关闭。
设计目标（docs/agent-program-wave3.md 第 2、3.2、5 节）：
```

```
    大模型是可选规划增强，而不是控制面的单点依赖。模型不可用、超时、
    乱输出或越权时，确定性规则模式（现有 RulePlanner）继续运行，
    原有派单闭环必须继续工作。
公开接口（冻结，不可改名）：
    JsonModelClient       可注入传输协议（不绑定厂商），只接收已脱敏的
                          结构化请求，返回 JSON 对象或字符串。
    ModelStepProposal     模型提出的单步提案（只含 tool_name / arguments /
                          decision_summary / confidence）。
    ModelPlanProposal     规划结果：source="model" 或 source="rule_fallback"，
                          带结构化原因与错误码，可直接被 Runtime 消费。
    ModelAdapterPlanner   接收 任务 / 业务上下文 / 工具摘要 / 角色 / Schema，
                          校验模型输出，非法时回退 RulePlanner。
    ModelAdapterError / ModelUnavailableError / ModelTimeoutError /
    ModelSchemaError      适配层结构化异常（语义清晰，见各错误码）。
纪律（与 Harness 手册 3.7 解密预算一致）：
- 只允许模型输出 tool_name / arguments / decision_summary / confidence，
  其余字段（含 chain_of_thought / reasoning）一律拒绝。
- 未知工具、未允许参数、缺字段、非法类型、超长字段、敏感工具越权
  → 回退规则规划（可观测的明确结果，绝不假装模型成功）。
- 超时 / 连接失败 / HTTP 错误 / JSON 解析失败 → 统一回退规则规划。
- 日志只记录 摘要 / 耗时 / 响应哈希 / 错误码，绝不记录完整 Prompt
  或模型原始回复。
- 配置关闭时完全不调用模型客户端（默认关闭）。
- 模型输出只作为「待执行提案」（is_proposal=True），任何情况下
  都不代表事实；必须经过严格 Schema、工具存在性、角色权限和
  风险级别校验后才可进入执行计划。
"""
from __future__ import annotations
import json
import logging
import socket
import time
from dataclasses import dataclass
from typing import Any, Callable, Protocol
from urllib.error import HTTPError, URLError
from urllib.request import Request, urlopen
from app.services.agents.errors import AgentError
from app.services.agents.model import AgentRunRequest, RiskLevel, RuleStep, RuntimeConfig
from app.services.agents.planner import RulePlanner
from app.services.agents.schema import canonical_json, sha256_hex, validate_schema
# ---------------------------------------------------------------------------
# 模块级常量
# ---------------------------------------------------------------------------
# 结果来源标记
SOURCE_MODEL = "model"
SOURCE_RULE_FALLBACK = "rule_fallback"
# 适配层结构化错误码（适配层专用，不属于手册 3.3 的 Runtime 冻结错误码；
# 集成后也不会写入 run 的 termination_reason / step.error_code）
MODEL_DISABLED = "model_disabled"                  # 配置关闭（默认）
MODEL_NOT_CONFIGURED = "model_not_configured"      # 未注入 client
```

```
MODEL_TIMEOUT = "model_timeout"                    # 模型调用超时



MODEL_CONNECTION_FAILED = "model_connection_failed"  # 连接失败
MODEL_HTTP_ERROR = "model_http_error"              # HTTP 错误
MODEL_INVALID_JSON = "model_invalid_json"          # JSON 解析失败
MODEL_SCHEMA_ERROR = "model_schema_error"          # 缺字段 / 类型错 / 超长 / 顶层结构错
MODEL_UNKNOWN_TOOL = "model_unknown_tool"          # 未知工具
MODEL_DISALLOWED_ARGUMENT = "model_disallowed_argument"  # 参数不在白名单 / 不满足工具 Schema
MODEL_ROLE_DENIED = "model_role_denied"            # 角色越权（含敏感工具越权）
MODEL_FORBIDDEN_FIELD = "model_forbidden_field"    # 包含思维链等被禁止字段
MODEL_INTERNAL_ERROR = "model_internal_error"      # 客户端抛出意外异常
# 模型输出中永久禁止出现的字段名（大小写不敏感递归检测）。
# 注意：错误消息中不得回显这些字段名（日志与结果必须保持干净）。
FORBIDDEN_FIELD_NAMES: frozenset[str] = frozenset({"chain_of_thought", "reasoning"})
# 请求脱敏：payload 里遇到这些键直接丢弃（大小写不敏感），
# 同时覆盖敏感载荷与思维链字段，双保险。
SENSITIVE_KEYS: frozenset[str] = frozenset(
    {
        "api_key",
        "authorization",
        "token",
        "password",
        "secret",
        "private_key",
        "credential",
        "cookie",
        "session_id",
        "chain_of_thought",
        "reasoning",
    }
)
# 长度 / 数量上限（超限即按「超长字段 / 非法类型」拒绝并回退）
MAX_PLAN_STEPS = 10
MAX_TOOL_NAME_LEN = 64
MAX_SUMMARY_LEN = 200
MAX_ARGUMENTS_JSON_LEN = 1024
MAX_PROMPT_STR_LEN = 200
MAX_COLLECTION_ITEMS = 20
MAX_DEPTH = 3
MAX_SCHEMA_STR_LEN = 500  # Schema 是契约：只截断超长描述，不做深度/数量截断
# 单个模型步骤的严格 Schema（4 个字段全必填；additionalProperties
# 语义由代码显式强制，因为 validate_schema 不支持该关键字）
STEP_SCHEMA: dict[str, Any] = {
    "type": "object",
    "required": ["tool_name", "arguments", "decision_summary", "confidence"],
    "properties": {
        "tool_name": {"type": "string", "minLength": 1, "maxLength": MAX_TOOL_NAME_LEN},
        "arguments": {"type": "object"},
        "decision_summary": {"type": "string", "maxLength": MAX_SUMMARY_LEN},
        "confidence": {"type": "number", "minimum": 0.0, "maximum": 1.0},
    },
```

```
}


# 模型输出顶层严格 Schema（只允许 plan 列表）
DEFAULT_MODEL_SCHEMA: dict[str, Any] = {
    "type": "object",
    "required": ["plan"],
    "properties": {
        "plan": {
            "type": "array",
            "minItems": 1,
            "maxItems": MAX_PLAN_STEPS,
            "items": STEP_SCHEMA,
        }
    },
}
_LOGGER = logging.getLogger("oceanus.agents.model_adapter")
# ---------------------------------------------------------------------
# 适配层结构化错误（冻结公开类名）
# ---------------------------------------------------------------------
class ModelAdapterError(Exception):
    """模型适配层异常基类。
    `code` 取本模块的 MODEL_* 错误码；错误只用于「回退规则规划」的
    可观测原因，绝不进入 Runtime 的冻结错误码集合。
    """
    def __init__(self, message: str, *, code: str = MODEL_INTERNAL_ERROR) -> None:
        self.code = code
        super().__init__(message)
class ModelUnavailableError(ModelAdapterError):
    """模型不可用：连接失败 / 服务不可达 / 未配置传输。"""
    def __init__(self, message: str, *, code: str = MODEL_CONNECTION_FAILED) -> None:
        super().__init__(message, code=code)
class ModelTimeoutError(ModelAdapterError):
    """模型调用超时。"""
    def __init__(self, message: str, *, code: str = MODEL_TIMEOUT) -> None:
        super().__init__(message, code=code)
class ModelSchemaError(ModelAdapterError):
    """模型输出未通过严格校验：非法 JSON / 缺字段 / 类型错 / 超长 /
    未知工具 / 未允许参数 / 角色越权 / 含被禁止字段。"""
    def __init__(self, message: str, *, code: str = MODEL_SCHEMA_ERROR) -> None:
        super().__init__(message, code=code)
# ---------------------------------------------------------------------
# 可注入传输协议（冻结公开类名，不绑定厂商）
# ---------------------------------------------------------------------
class JsonModelClient(Protocol):
    """可注入模型传输协议（冻结）。
    只接收已脱敏的结构化请求（dict），返回 JSON 对象（dict）或
    JSON 字符串。不绑定具体厂商；任何真实网络实现都必须默认关闭
    （由调用方控制开关），测试只能注入假客户端，不得访问公网。
    实现约定（由实现方保证）：
    - 超时抛 `ModelTimeoutError`；
    - 连接失败 / 服务不可达抛 `ModelUnavailableError`；
```

```
    - HTTP / 传输错误抛 `ModelAdapterError`（可带 code）；

    - 其余一律返回 dict 或 str。
    """
    def complete(self, payload: dict[str, Any]) -> dict[str, Any] | str:
        if not isinstance(payload, dict):
            raise TypeError(f"payload 必须是 dict：{type(payload).__name__}")
        raise NotImplementedError("JsonModelClient.complete 未实现")
# ---------------------------------------------------------------------
# OpenAI-compatible Chat Completions 传输（默认不装配）
# ---------------------------------------------------------------------
DEFAULT_MODEL_SYSTEM_PROMPT = (
    "You are the planning component of a marine-litter dispatch agent. "
    "Use only the tools and argument schemas supplied by the user. "
    "Return one JSON object that matches output_schema exactly. "
    "Do not include markdown, commentary, reasoning, or chain-of-thought."
)
class OpenAICompatibleModelClient:
    """OpenAI-compatible Chat Completions transport.
    This is a deploy-time adapter, not a default dependency. It only runs
    when the caller explicitly enables the model adapter and provides a
    base URL plus model name. Tests inject a fake opener and never use the
    network.
    """
    def __init__(
        self,
        *,
        base_url: str,
        model: str,
        api_key: str = "",
        timeout_ms: int = 5000,
        max_output_tokens: int = 1024,
        system_prompt: str = DEFAULT_MODEL_SYSTEM_PROMPT,
        opener: Callable[..., Any] | None = None,
    ) -> None:
        endpoint = str(base_url or "").strip().rstrip("/")
        model_name = str(model or "").strip()
        if not endpoint:
            raise ValueError("base_url 不能为空")
        if not model_name:
            raise ValueError("model 不能为空")
        if timeout_ms <= 0:
            raise ValueError("timeout_ms 必须大于 0")
        if max_output_tokens <= 0:
            raise ValueError("max_output_tokens 必须大于 0")
        if endpoint.endswith("/chat/completions"):
            request_url = endpoint
        else:
            request_url = f"{endpoint}/chat/completions"
        self.base_url = endpoint
        self.request_url = request_url
```

```
        self.model = model_name



        self.api_key = str(api_key or "").strip()
        self.timeout_ms = int(timeout_ms)
        self.max_output_tokens = int(max_output_tokens)
        self.system_prompt = str(system_prompt)
        self._opener = opener or urlopen
    def complete(self, payload: dict[str, Any]) -> str:
        """Send a sanitized planner payload and return the model JSON text."""
        if not isinstance(payload, dict):
            raise TypeError(f"payload 必须是 dict：{type(payload).__name__}")
        body = json.dumps(
            {
                "model": self.model,
                "messages": [
                    {"role": "system", "content": self.system_prompt},
                    {
                        "role": "user",
                        "content": json.dumps(
                            payload,
                            ensure_ascii=False,
                            separators=(",", ":"),
                            sort_keys=True,
                        ),
                    },
                ],
                "temperature": 0,
                "max_tokens": self.max_output_tokens,
                "response_format": {"type": "json_object"},
            },
            ensure_ascii=False,
            separators=(",", ":"),
        ).encode("utf-8")
        headers = {
            "Accept": "application/json",
            "Content-Type": "application/json",
            "User-Agent": "Oceanus-Agent/1.0",
        }
        if self.api_key:
            headers["Authorization"] = f"Bearer {self.api_key}"
        request = Request(
            self.request_url,
            data=body,
            headers=headers,
            method="POST",
        )
        try:
            with self._opener(request, timeout=self.timeout_ms / 1000) as response:
                raw_response = response.read()
        except HTTPError as exc:
            raise ModelAdapterError(
```

```
                f"模型服务返回 HTTP {exc.code}",
                code=MODEL_HTTP_ERROR,
            ) from exc
        except (TimeoutError, socket.timeout) as exc:
            raise ModelTimeoutError("模型调用超时") from exc
        except URLError as exc:
            if isinstance(exc.reason, (TimeoutError, socket.timeout)):
                raise ModelTimeoutError("模型调用超时") from exc
            raise ModelUnavailableError("模型服务连接失败") from exc
        except OSError as exc:
            raise ModelUnavailableError("模型服务连接失败") from exc
        except ValueError as exc:
            raise ModelUnavailableError("模型服务地址配置无效") from exc
        if isinstance(raw_response, bytes):
            try:
                response_text = raw_response.decode("utf-8")
            except UnicodeDecodeError as exc:
                raise ModelAdapterError(
                    "模型服务响应不是 UTF-8",
                    code=MODEL_INVALID_JSON,
                ) from exc
        elif isinstance(raw_response, str):
            response_text = raw_response
        else:
            raise ModelAdapterError(
                "模型服务响应类型非法",
                code=MODEL_INVALID_JSON,
            )
        try:
            envelope = json.loads(response_text)
        except json.JSONDecodeError as exc:
            raise ModelAdapterError(
                "模型服务响应不是合法 JSON",
                code=MODEL_INVALID_JSON,
            ) from exc
        content = self._extract_content(envelope)
        if isinstance(content, str):
            return content
        if isinstance(content, dict):
            return json.dumps(content, ensure_ascii=False, separators=(",", ":"))
        raise ModelSchemaError(
            "模型服务响应缺少 message.content",
            code=MODEL_SCHEMA_ERROR,
        )
    @staticmethod
    def _extract_content(envelope: Any) -> Any:
        if not isinstance(envelope, dict):
            raise ModelSchemaError(
                "模型服务响应必须是 JSON 对象",
                code=MODEL_SCHEMA_ERROR,
```

```
            )



        choices = envelope.get("choices")
        if not isinstance(choices, list) or not choices:
            raise ModelSchemaError(
                "模型服务响应缺少 choices",
                code=MODEL_SCHEMA_ERROR,
            )
        first = choices[0]
        if not isinstance(first, dict):
            raise ModelSchemaError(
                "模型服务 choices[0] 必须是对象",
                code=MODEL_SCHEMA_ERROR,
            )
        message = first.get("message")
        if isinstance(message, dict):
            content = message.get("content")
        else:
            content = first.get("text")
        if isinstance(content, str):
            return content
        if isinstance(content, list):
            parts: list[str] = []
            for item in content:
                if isinstance(item, dict) and isinstance(item.get("text"), str):
                    parts.append(item["text"])
            return "".join(parts)
        return content
# ---------------------------------------------------------------------
# 提案对象（冻结公开类名）
# ---------------------------------------------------------------------
@dataclass(frozen=True)
class ModelStepProposal:
    """模型提出的单步提案（只含 4 个结构化字段，冻结）。
    只承载决策摘要与工具输入，不承载思维链；`arguments` 必须已通过
    工具白名单与工具 Schema 校验。
    """
    tool_name: str
    arguments: dict[str, Any]
    decision_summary: str = ""
    confidence: float = 0.0
@dataclass(frozen=True)
class ModelPlanProposal:
    """一次规划的结果（source 二选一，冻结）。
    - source="model"：模型输出通过全部校验后转化的可执行计划；
      `model_steps` 为校验通过的原始提案，`steps` 为 Runtime 可消费的
      RuleStep 列表。模型输出只作为待执行提案（is_proposal=True），
      不代表事实。
    - source="rule_fallback"：模型不可用 / 超时 / 乱输出 / 越权时
      确定性回退 `RulePlanner` 的规则计划；`error_code` 与
       `fallback_reason` 给出结构化、可观测的回退原因，
```

绝不假装模型成功。

```
    """
    source: str  # SOURCE_MODEL | SOURCE_RULE_FALLBACK
    steps: tuple[RuleStep, ...]
    model_steps: tuple[ModelStepProposal, ...] = ()
    decision_summary: str = ""
    confidence: float = 0.0
    fallback_reason: str | None = None
    error_code: str | None = None
    latency_ms: int = 0
    response_hash: str | None = None
    is_proposal: bool = True  # 模型输出只是提案，任何情况下都不是事实
# ---------------------------------------------------------------------
# 辅助：把 ToolRegistry 压成脱敏的工具摘要（供 payload 与校验共用）
# ---------------------------------------------------------------------
def summarize_tools(registry: Any) -> list[dict[str, Any]]:
    """把 ToolRegistry 压缩为脱敏工具摘要列表（无 handler、无真实数据）。
    摘要字段：name / version / description / risk_level / timeout_ms /
    idempotent / allowed_roles / input_schema。供 ModelAdapterPlanner
    做工具存在性与参数白名单校验，也作为发给模型的工具描述。
    """
    summary: list[dict[str, Any]] = []
    for tool in registry.list():
        risk = tool.risk_level
        summary.append(
            {
                "name": tool.name,
                "version": tool.version,
                "description": tool.description,
                "risk_level": risk.value if isinstance(risk, RiskLevel) else str(risk),
                "timeout_ms": tool.timeout_ms,
                "idempotent": bool(tool.idempotent),
                "allowed_roles": list(tool.allowed_roles),
                "input_schema": tool.input_schema,
            }
        )
    return summary
# ---------------------------------------------------------------------
# 规划适配器（冻结公开类名）
# ---------------------------------------------------------------------
class ModelAdapterPlanner:
    """模型规划适配器：模型优先、规则兜底。
    接收 任务（AgentRunRequest） / 业务上下文（dict，已脱敏） /
    工具摘要（list[dict]） / 角色（str） / Schema（dict，缺省用严格默认）。
    行为：
    1. 配置关闭（默认）或未注入 client → 不调用模型，直接回退规则规划。
    2. 调用 client（只传已脱敏 payload）→ 解析 JSON → 严格校验 →
       转化为 RuleStep 列表。
    3. 任何失败（超时 / 连接 / HTTP / 非法 JSON / Schema / 未知工具 /
       越权 / 被禁止字段）→ 回退 RulePlanner，结果带 source=
```

```
    "rule_fallback" 与结构化原因。
    """
    def __init__(
        self,
        *,
        client: JsonModelClient | None = None,
        enabled: bool = False,
        rule_planner: RulePlanner | None = None,
        config: RuntimeConfig | None = None,
        timeout_ms: int = 5000,
        max_steps: int = MAX_PLAN_STEPS,
        blocked_tools: tuple[str, ...] = (),
        clock: Any = None,
        log: logging.Logger | None = None,
    ) -> None:
        self.client = client
        self.enabled = bool(enabled)
        self.config = config or RuntimeConfig()
        self.rule_planner = rule_planner or RulePlanner(self.config)
        self.timeout_ms = int(timeout_ms)
        self.max_steps = int(max_steps)
        self.blocked_tools = frozenset(blocked_tools)
        self.clock = clock if clock is not None else _MonotonicClock()
        self.logger = log if log is not None else _LOGGER
    # ------------------------------------------------------------------
    # 公共入口
    # ------------------------------------------------------------------
    def plan(
        self,
        task: AgentRunRequest,
        business_context: dict[str, Any] | None = None,
        tool_summary: list[dict[str, Any]] | None = None,
        role: str | None = None,
        schema: dict[str, Any] | None = None,
    ) -> ModelPlanProposal:
        """规划入口：返回 ModelPlanProposal（model 或 rule_fallback）。
        - task：触发 Agent 运行的请求（AgentRunRequest）。
        - business_context：触发源脱敏后的业务上下文（本模块会再次脱敏）。
        - tool_summary：已注册工具摘要（summarize_tools 产出或等价结构）。
        - role：发起人角色；缺省取 task.role。
        - schema：模型输出严格 Schema；缺省 DEFAULT_MODEL_SCHEMA。
        """
        if not isinstance(task, AgentRunRequest):
            raise TypeError(f"task 必须是 AgentRunRequest：{type(task).__name__}")
        started = self.clock.now_ms()
        effective_role = role if role is not None else task.role
        if not isinstance(effective_role, str) or not effective_role:
            raise TypeError(f"role 必须是非空 str：{effective_role!r}")
        effective_schema = schema if schema is not None else DEFAULT_MODEL_SCHEMA
        tools = list(tool_summary or [])
```

```
        tool_map = {
            str(item.get("name")): item for item in tools if isinstance(item, dict) and item.get("n…
        }
        # 1. 配置关闭 / 未注入 client：完全不调用模型客户端
        if not self.enabled:
            return self._fallback(
                task, MODEL_DISABLED, "模型适配层配置关闭（默认），回退规则规划", started
            )
        if self.client is None:
            return self._fallback(
                task, MODEL_NOT_CONFIGURED, "未注入 JsonModelClient，无法调用模型，回退规则规划", s…
            )
        # 2. 调用模型（只传已脱敏 payload）
        payload = self._build_payload(task, business_context or {}, tools, effective_role, effectiv…
        try:
            raw = self.client.complete(payload)
        except ModelTimeoutError as exc:
            return self._fallback(task, exc.code or MODEL_TIMEOUT, "模型调用超时", started)
        except ModelUnavailableError as exc:
            return self._fallback(task, exc.code or MODEL_CONNECTION_FAILED, "模型不可用", started)
        except ModelAdapterError as exc:
            return self._fallback(task, exc.code or MODEL_HTTP_ERROR, "模型传输/服务错误", started)
        except TimeoutError as exc:
            return self._fallback(task, MODEL_TIMEOUT, "模型调用超时（内置超时异常）", started)
        except (ConnectionError, OSError) as exc:
            return self._fallback(task, MODEL_CONNECTION_FAILED, "模型连接失败", started)
        except Exception as exc:  # noqa: BLE001：客户端意外异常统一回退，不让规划崩溃
            self.logger.warning(
                "[ModelAdapter] source=rule_fallback code=%s exc_type=%s",
                MODEL_INTERNAL_ERROR,
                type(exc).__name__,
            )
            return self._fallback(task, MODEL_INTERNAL_ERROR, "模型客户端抛出意外异常", started)
        # 3. 解析与严格校验（失败 → 回退）
        response_hash = sha256_hex(raw)
        try:
            parsed = self._parse_response(raw)
        except ModelSchemaError as exc:
            return self._fallback(task, exc.code, "模型输出 JSON 解析失败", started)
        try:
            model_steps = self._validate_output(parsed, tool_map, effective_role, effective_schema)
        except ModelSchemaError as exc:
            return self._fallback(task, exc.code, str(exc), started)
        # 4. 校验通过 → 转化为 Runtime 可消费的 RuleStep 计划
        rule_steps = tuple(self._to_rule_step(step) for step in model_steps)
        latency = self.clock.now_ms() - started
        summary = "模型计划 {n} 步 ({tools}) ".format(
            n=len(rule_steps), tools=", ".join(s.tool for s in rule_steps[:5])
        )
        self.logger.info(
```

```
            "[ModelAdapter] source=model steps=%d latency=%dms response_hash=%s",
            len(rule_steps),
            latency,
            response_hash,
        )
        return ModelPlanProposal(
            source=SOURCE_MODEL,
            steps=rule_steps,
            model_steps=model_steps,
            decision_summary=summary,
            confidence=sum(s.confidence for s in model_steps) / len(model_steps),
            latency_ms=latency,
            response_hash=response_hash,
        )
    # ------------------------------------------------------------------
    # 回退（可观测的明确结果，绝不假装模型成功）
    # ------------------------------------------------------------------
    def _fallback(
        self,
        task: AgentRunRequest,
        code: str,
        detail: str,
        started_ms: int,
    ) -> ModelPlanProposal:
        latency = self.clock.now_ms() - started_ms
        try:
            rule_steps = self.rule_planner.build(task)
        except AgentError as exc:
            # 规则计划本身配置错误（与 Runtime 行为一致地暴露，不让兜底静默空转）
            raise ModelAdapterError(
                f"规则规划器回退失败：{exc}", code=MODEL_INTERNAL_ERROR
            ) from exc
        self.logger.info(
            "[ModelAdapter] source=rule_fallback code=%s latency=%dms steps=%d",
            code,
            latency,
            len(rule_steps),
        )
        return ModelPlanProposal(
            source=SOURCE_RULE_FALLBACK,
            steps=tuple(rule_steps),
            model_steps=(),
            decision_summary="规则兜底 {n} 步（原因 {code}）".format(n=len(rule_steps), code=code),
            confidence=0.0,
            fallback_reason=f"{code}: {detail}",
            error_code=code,
            latency_ms=latency,
            response_hash=None,
        )
    # ------------------------------------------------------------------
```

```
    # 请求构建（只发已脱敏的结构化请求）


    # ------------------------------------------------------------------
    def _build_payload(
        self,
        task: AgentRunRequest,
        business_context: dict[str, Any],
        tool_summary: list[dict[str, Any]],
        role: str,
        schema: dict[str, Any],
    ) -> dict[str, Any]:
        return {
            "task": {
                "trigger_type": task.trigger_type,
                "objective": self._clip_text(task.objective),
                "role": role,
                "param_keys": sorted((task.params or {}).keys()),
                "params": self._sanitize(task.params or {}),
            },
            "business_context": self._sanitize(business_context),
            "tools": [self._summarize_tool(item) for item in tool_summary],
            "output_schema": self._sanitize_schema(schema),
            "hints": {"timeout_ms": self.timeout_ms, "max_steps": self.max_steps},
        }
    def _summarize_tool(self, entry: dict[str, Any]) -> dict[str, Any]:
        return {
            "name": str(entry.get("name") or ""),
            "version": str(entry.get("version") or ""),
            "description": self._clip_text(str(entry.get("description") or "")),
            "risk_level": str(entry.get("risk_level") or ""),
            "timeout_ms": entry.get("timeout_ms"),
            "idempotent": bool(entry.get("idempotent")),
            "allowed_roles": [str(r) for r in (entry.get("allowed_roles") or ())],
            "input_schema": self._sanitize_schema(entry.get("input_schema") or {}),
        }
    def _sanitize(self, value: Any, depth: int = 0) -> Any:
        """递归脱敏：丢弃敏感键、截断长字符串、限制集合大小与深度。"""
        if depth > MAX_DEPTH:
            return "<depth>"
        if isinstance(value, str):
            return self._clip_text(value)
        if isinstance(value, dict):
            out: dict[str, Any] = {}
            for key, sub in value.items():
                if not isinstance(key, str) or key.lower() in SENSITIVE_KEYS:
                    continue
                if len(out) >= MAX_COLLECTION_ITEMS:
                    out["<truncated>"] = True
                    break
                out[key] = self._sanitize(sub, depth + 1)
            return out
```

```
        if isinstance(value, (list, tuple)):
            items = [self._sanitize(item, depth + 1) for item in value[:MAX_COLLECTION_ITEMS]]
            if len(value) > MAX_COLLECTION_ITEMS:
                items.append("<truncated>")
            return items
        if value is None or isinstance(value, (bool, int, float)):
            return value
        return f"<{type(value).__name__}>"
    @staticmethod
    def _clip_text(text: str, limit: int = MAX_PROMPT_STR_LEN) -> str:
        if not isinstance(text, str):
            text = str(text)
        return text if len(text) <= limit else text[: limit - 3] + "..."
    def _sanitize_schema(self, value: Any) -> Any:
        """Schema 是契约：只做敏感键清洗与超长字符串截断。
        不做深度 / 数量截断，保证发给模型的 output_schema 与
        本地校验所用的 Schema 完全一致。
        """
        if isinstance(value, str):
            return self._clip_text(value, limit=MAX_SCHEMA_STR_LEN)
        if isinstance(value, dict):
            return {
                key: self._sanitize_schema(sub)
                for key, sub in value.items()
                if isinstance(key, str) and key.lower() not in SENSITIVE_KEYS
            }
        if isinstance(value, (list, tuple)):
            return [self._sanitize_schema(item) for item in value]
        if value is None or isinstance(value, (bool, int, float)):
            return value
        return f"<{type(value).__name__}>"
    # ------------------------------------------------------------------
    # 解析与严格校验
    # ------------------------------------------------------------------
    @staticmethod
    def _parse_response(raw: dict[str, Any] | str) -> Any:
        """客户端返回 dict 或 str；str 必须能解析为 JSON。"""
        if isinstance(raw, dict):
            return raw
        if isinstance(raw, str):
            text = raw.strip()
            if not text:
                raise ModelSchemaError("模型返回空响应", code=MODEL_INVALID_JSON)
            try:
                return json.loads(text)
            except json.JSONDecodeError as exc:
                raise ModelSchemaError(
                    "模型返回内容不是合法 JSON",
                    code=MODEL_INVALID_JSON,
                ) from exc
```

```
        raise ModelSchemaError(
            f"模型返回类型必须是 dict 或 str，实际 {type(raw).__name__}",
            code=MODEL_SCHEMA_ERROR,
        )
    def _validate_output(
        self,
        parsed: Any,
        tool_map: dict[str, dict[str, Any]],
        role: str,
        schema: dict[str, Any],
    ) -> tuple[ModelStepProposal, ...]:
        self._reject_forbidden_fields(parsed)
        if not isinstance(parsed, dict):
            raise ModelSchemaError("模型输出必须是 JSON 对象", code=MODEL_SCHEMA_ERROR)
        extra = set(parsed) - {"plan"}
        if extra:
            raise ModelSchemaError(
                f"模型输出顶层含未允许字段：{sorted(extra)}",
                code=MODEL_SCHEMA_ERROR,
            )
        schema_errors = validate_schema(parsed, schema)
        if schema_errors:
            raise ModelSchemaError(
                "模型输出未通过严格 Schema：" + "; ".join(schema_errors),
                code=MODEL_SCHEMA_ERROR,
            )
        plan = parsed["plan"]
        if len(plan) > self.max_steps:
            raise ModelSchemaError(
                f"模型计划步数 {len(plan)} 超过上限 {self.max_steps}",
                code=MODEL_SCHEMA_ERROR,
            )
        return tuple(self._validate_step(item, tool_map, role) for item in plan)
    def _validate_step(
        self,
        step: Any,
        tool_map: dict[str, dict[str, Any]],
        role: str,
    ) -> ModelStepProposal:
        if not isinstance(step, dict):
            raise ModelSchemaError("模型步骤必须是 JSON 对象", code=MODEL_SCHEMA_ERROR)
        self._reject_forbidden_fields(step)
        extra = set(step) - {"tool_name", "arguments", "decision_summary", "confidence"}
        if extra:
            raise ModelSchemaError(
                f"模型步骤含未允许字段：{sorted(extra)}",
                code=MODEL_SCHEMA_ERROR,
            )
        schema_errors = validate_schema(step, STEP_SCHEMA)
        if schema_errors:
```

```
            raise ModelSchemaError(
                "模型步骤未通过严格 Schema: " + "; ".join(schema_errors),
                code=MODEL_SCHEMA_ERROR,
            )
        tool_name = step["tool_name"]
        entry = tool_map.get(tool_name)
        if entry is None:
            raise ModelSchemaError(f"未知工具: {tool_name}", code=MODEL_UNKNOWN_TOOL)
        if tool_name in self.blocked_tools:
            raise ModelSchemaError(f"工具被策略禁止: {tool_name}", code=MODEL_ROLE_DENIED)
        # 角色权限 + 风险级别校验 (敏感工具越权在此拒绝)
        allowed_roles = tuple(entry.get("allowed_roles") or ())
        if role not in allowed_roles:
            risk = str(entry.get("risk_level") or "")
            prefix = "敏感/高风险工具越权" if risk in ("sensitive", "device_command") else "角色越...
            raise ModelSchemaError(
                f"{prefix}: 角色 {role} 无权调用 {tool_name} (允许 {list(allowed_roles)}) ",
                code=MODEL_ROLE_DENIED,
            )
        risk_level = entry.get("risk_level")
        if risk_level is not None and str(risk_level) not in {
            "read_only", "write", "device_command", "sensitive",
        }:
            raise ModelSchemaError(
                f"工具 {tool_name} 声明了未知风险级别: {risk_level}",
                code=MODEL_SCHEMA_ERROR,
            )
        # 参数白名单 + 工具 Schema 校验 (只允许模型提出白名单参数)
        arguments = step["arguments"]
        input_schema = entry.get("input_schema")
        properties = input_schema.get("properties") if isinstance(input_schema, dict) else None
        if not isinstance(properties, dict):
            raise ModelSchemaError(
                f"工具 {tool_name} 缺少参数白名单 (input_schema.properties) ",
                code=MODEL_DISALLOWED_ARGUMENT,
            )
        unknown_args = sorted(key for key in arguments if key not in properties)
        if unknown_args:
            raise ModelSchemaError(
                f"参数不在工具白名单: {tool_name} -> {unknown_args}",
                code=MODEL_DISALLOWED_ARGUMENT,
            )
        arg_errors = validate_schema(arguments, input_schema)
        if arg_errors:
            raise ModelSchemaError(
                f"参数未通过工具 Schema: {tool_name} -> " + "; ".join(arg_errors),
                code=MODEL_DISALLOWED_ARGUMENT,
            )
        if len(canonical_json(arguments)) > MAX_ARGUMENTS_JSON_LEN:
            raise ModelSchemaError(
```

```
                f"工具 {tool_name} 参数过大（超过 {MAX_ARGUMENTS_JSON_LEN} 字符）",
                code=MODEL_SCHEMA_ERROR,
            )
        return ModelStepProposal(
            tool_name=tool_name,
            arguments=dict(arguments),
            decision_summary=step.get("decision_summary") or "",
            confidence=float(step.get("confidence") or 0.0),
        )
    @staticmethod
    def _reject_forbidden_fields(value: Any) -> None:
        """递归拒绝 chain_of_thought / reasoning 字段（大小写不敏感）。
        错误消息不回显字段名，保证日志与结果中不出现思维链字段。
        """
        if isinstance(value, dict):
            for key, sub in value.items():
                if isinstance(key, str) and key.lower() in FORBIDDEN_FIELD_NAMES:
                    raise ModelSchemaError(
                        "模型输出包含被禁止字段（思维链/推理）",
                        code=MODEL_FORBIDDEN_FIELD,
                    )
                ModelAdapterPlanner._reject_forbidden_fields(sub)
        elif isinstance(value, (list, tuple)):
            for item in value:
                ModelAdapterPlanner._reject_forbidden_fields(item)
    # ------------------------------------------------------------------
    # 转化为 Runtime 可消费的计划
    # ------------------------------------------------------------------
    @staticmethod
    def _to_rule_step(step: ModelStepProposal) -> RuleStep:
        """把模型提案转化为 RuleStep（参数已具体绑定，expect=None）。
        Runtime 的 _stage_execute 会再次执行工具 Schema / 策略守卫 /
        审批，因此模型参数即使通过本层校验，仍受确定性控制面约束。
        """
        return RuleStep(
            tool=step.tool_name,
            input=dict(step.arguments),
            summary=step.decision_summary or f"模型提议调用 {step.tool_name}",
            expect=None,
            on_error="terminate",
        )
class _MonotonicClock:
    """单调毫秒时钟（默认；测试可注入 FakeClock 保证确定性）。"""
    def now_ms(self) -> int:
        return int(time.monotonic() * 1000)
__all__ = [
    # 冻结公开接口（docs/agent-program-wave3.md 3.2）
    "JsonModelClient",
    "OpenAICompatibleModelClient",
    "ModelStepProposal",
```

```
    "ModelPlanProposal",
    "ModelAdapterPlanner",
    "ModelAdapterError",
    "ModelUnavailableError",
    "ModelTimeoutError",
    "ModelSchemaError",
    # 扩展
    "summarize_tools",
    "DEFAULT_MODEL_SCHEMA",
    "STEP_SCHEMA",
    "SOURCE_MODEL",
    "SOURCE_RULE_FALLBACK",
    "DEFAULT_MODEL_SYSTEM_PROMPT",
]
# ===== backend/app/services/agents/policy.py =====
"""策略守卫：确定性规则策略，代替模型做基线决策。
手册 5.6：规则策略先给出可验证的基线决策，大模型只做增强；
`runtime_config.model_available=False` 时规则模式继续工作。
`PolicyGuard` 是协议（可注入自定义实现），`RulePolicyGuard` 是首版
确定性实现：
- `check_tool`：角色必须在工具 allowed_roles 内；risk_level 达到
  `require_approval_risk` 阈值 → `requires_approval=True`（挂起审批）。
- `check_plan`：对整个计划的工具可用性与角色权限做聚合预检，
  产出计划级 PolicyDecision（记录 policy_version）。
"""
from __future__ import annotations
from dataclasses import dataclass, field
from typing import TYPE_CHECKING, Any, Protocol
from app.services.agents.model import AgentRunRequest, RiskLevel, RuntimeConfig
if TYPE_CHECKING:
    from app.services.agents.tools import ToolDefinition, ToolRegistry
    from app.services.agents.model import RuleStep
@dataclass(frozen=True)
class PolicyDecision:
    """策略守卫的判定结果。
    - allow：是否放行
    - reason：业务理由（审计留痕，解密预算允许保存）
    - requires_approval：是否必须人工审批
    - risk_level：被判定动作的风险级别
    - policy_version：判定所用策略版本
    """
    allow: bool
    reason: str = ""
    requires_approval: bool = False
    risk_level: RiskLevel | None = None
    policy_version: str = ""
    @classmethod
    def allow_decision(
        cls,
        *,
```

```
        violations = validate_memory_content(entry.content)
        if violations:
            raise AgentError(
                f"记忆内容包含禁止字段 {violations}：解密预算只允许保存决策摘要、"
                "工具输入摘要、输出摘要与业务理由",
                code="internal_error",
            )
        if entry.memory_type not in MEMORY_TYPES:
            raise AgentError(
                f"非法记忆类型 {entry.memory_type}，允许 {list(MEMORY_TYPES)}",
                code="internal_error",
            )
        if not entry.memory_id:
            if self._id_factory is not None:
                entry.memory_id = self._id_factory.new("mem_")
            else:
                import uuid
                entry.memory_id = f"mem_{uuid.uuid4().hex[:12]}"
        if entry.created_at is None:
            entry.created_at = (
                self._clock.now()
                if self._clock is not None
                else datetime.now(timezone.utc)
            )
        if entry.memory_id in self._entries:
            raise TaskConflictError(f"memory_id 重复：{entry.memory_id}")
        self._entries[entry.memory_id] = entry
        return entry
    def get(self, memory_id: str) -> MemoryEntry | None:
        return self._entries.get(memory_id)
    def find(
        self,
        *,
        scope_type: str | None = None,
        scope_id: str | None = None,
        memory_type: str | None = None,
    ) -> list[MemoryEntry]:
        result = list(self._entries.values())
        if scope_type is not None:
            result = [e for e in result if e.scope_type == scope_type]
        if scope_id is not None:
            result = [e for e in result if e.scope_id == scope_id]
        if memory_type is not None:
            result = [e for e in result if e.memory_type == memory_type]
        result.sort(key=lambda e: _time_key(e.created_at))
        return result
    def __len__(self) -> int:
        return len(self._entries)
# ===== backend/app/services/agents/schema.py =====
"""最小 JSON Schema 校验器与确定性哈希。
```

为什么自研而不是引 jsonschema：

- 内核不允许依赖外部基础设施/模型，依赖也必须可离线、可确定性复现；
- 环境未安装 jsonschema（验收命令同样不安装任何新依赖）；
- 工具契约只需要实用子集：type / properties / required / items / enum / const / minLength / maxLength / pattern / minimum / maximum / minItems / maxItems。超出子集的关键字按 JSON Schema 语义忽略（宽松向前兼容）。

哈希统一用 `sha256_hex(canonical_json(value))`：
同一输入在任何机器/进程上产出同一哈希，供幂等键与轨迹摘要使用。
"""

```
from __future__ import annotations
import hashlib
import json
import re
from typing import Any
def canonical_json(value: Any) -> str:
    """把任意可 JSON 序列化对象压成确定性字符串（键排序、紧凑分隔）。"""
    return json.dumps(
        value,
        sort_keys=True,
        ensure_ascii=False,
        separators=(",", ":"),
        default=str,
    )
def sha256_hex(value: Any) -> str:
    """对值求确定性 sha256（输入/输出哈希，轨迹摘要用）。"""
    return hashlib.sha256(canonical_json(value).encode("utf-8")).hexdigest()
def _type_name(value: Any) -> str:
    """JSON 语义的类型名（bool 不是 integer）。"""
    if value is None:
        return "null"
    if isinstance(value, bool):
        return "boolean"
    if isinstance(value, int):
        return "integer"
    if isinstance(value, float):
        return "number"
    if isinstance(value, str):
        return "string"
    if isinstance(value, (list, tuple)):
        return "array"
    if isinstance(value, dict):
        return "object"
    return type(value).__name__
def _type_ok(value: Any, type_name: str) -> bool:
    if type_name == "null":
        return value is None
    if type_name == "boolean":
        return isinstance(value, bool)
    if type_name == "string":
        return isinstance(value, str)
```

```
    if type_name == "integer":
        return isinstance(value, int) and not isinstance(value, bool)
    if type_name == "number":
        return isinstance(value, (int, float)) and not isinstance(value, bool)
    if type_name == "array":
        return isinstance(value, (list, tuple))
    if type_name == "object":
        return isinstance(value, dict)
    return False
def validate_schema(value: Any, schema: dict[str, Any]) -> list[str]:
    """校验 value 是否满足 schema，返回错误列表；空列表表示通过。"""
    if not isinstance(schema, dict):
        return []
    errors: list[str] = []
    type_name = schema.get("type")
    if type_name is not None:
        if not _type_ok(value, str(type_name)):
            errors.append(f"期望类型 {type_name}，实际 {_type_name(value)}")
    if "enum" in schema and value not in schema["enum"]:
        errors.append(f"值不在枚举 {schema['enum']} 中")
    if "const" in schema and value != schema["const"]:
        errors.append(f"值不等于 const {schema['const']}")
    if isinstance(value, str):
        if "minLength" in schema and len(value) < schema["minLength"]:
            errors.append(f"字符串长度 {len(value)} 小于 minLength {schema['minLength']}")
        if "maxLength" in schema and len(value) > schema["maxLength"]:
            errors.append(f"字符串长度 {len(value)} 大于 maxLength {schema['maxLength']}")
        pattern = schema.get("pattern")
        if pattern is not None and re.search(str(pattern), value) is None:
            errors.append(f"字符串不匹配 pattern {pattern}")
    if isinstance(value, (int, float)) and not isinstance(value, bool):
        if "minimum" in schema and value < schema["minimum"]:
            errors.append(f"{value} 小于 minimum {schema['minimum']}")
        if "maximum" in schema and value > schema["maximum"]:
            errors.append(f"{value} 大于 maximum {schema['maximum']}")
    if isinstance(value, (list, tuple)):
        if "minItems" in schema and len(value) < schema["minItems"]:
            errors.append(f"数组长度 {len(value)} 小于 minItems {schema['minItems']}")
        if "maxItems" in schema and len(value) > schema["maxItems"]:
            errors.append(f"数组长度 {len(value)} 大于 maxItems {schema['maxItems']}")
        items_schema = schema.get("items")
        if isinstance(items_schema, dict):
            for i, item in enumerate(value):
                for e in validate_schema(item, items_schema):
                    errors.append(f"items[{i}]: {e}")
    if isinstance(value, dict):
        for req in schema.get("required", []):
            if req not in value:
                errors.append(f"缺少必填字段 {req}")
        properties = schema.get("properties", {})
```

```
        if isinstance(properties, dict):
            for key, sub in value.items():
                if key in properties:
                    for e in validate_schema(sub, properties[key]):
                        errors.append(f"{key}: {e}")
    return errors
def is_valid(value: Any, schema: dict[str, Any]) -> bool:
    """校验是否通过。"""
    return not validate_schema(value, schema)
# ===== backend/app/services/agents/model.py =====
"""Agent 内核领域模型：状态、步骤类型、风险级别、运行、步骤、结果。
字段名与 Harness 执行手册 3.7 数据表一一对齐，供 WP-02 ORM
直接映射，语义不允许改动：
    t_agent_run       -> AgentRun (run_id/trigger_type/objective/status/
                         policy_version/started_at/finished_at/
                         termination_reason/trace_id/created_at/updated_at)
    t_agent_step      -> AgentStep (step_id/run_id/step_no/step_type/
                         decision_summary/tool_name/tool_version/input_hash/
                         output_hash/status/latency_ms/error_code/created_at)
    t_agent_approval  -> ApprovalRecord (见 repositories.py)
    t_agent_memory    -> MemoryEntry (见 memory.py)
唯一约束 (手册 3.7)：
    run_id / step_id / memory_id / approval_id 唯一;
    (run_id, step_no) 唯一：由内存仓储在 add_step 时强制。
"""
from __future__ import annotations
import enum
from dataclasses import dataclass, field
from datetime import datetime
from typing import Any
# ---------------------------------------------------------------------------
# 冻结枚举 (手册 3.1 / 3.2 / 3.5)
# ---------------------------------------------------------------------------
class AgentStatus(enum.StrEnum):
    """运行状态 (手册 3.1，11 个值，禁止增删改)。
    终态为 succeeded / failed / cancelled / expired，终态不得再次迁移。
    """
    CREATED = "created"
    PLANNING = "planning"
    WAITING_POLICY = "waiting_policy"
    WAITING_APPROVAL = "waiting_approval"
    EXECUTING = "executing"
    OBSERVING = "observing"
    VERIFYING = "verifying"
    SUCCEEDED = "succeeded"
    FAILED = "failed"
    CANCELLED = "cancelled"
    EXPIRED = "expired"
class AgentStepType(enum.StrEnum):
    """步骤类型 (手册 3.2，8 个值，禁止增删改)。
```

每次重规划必须新增 replan 与后续 plan 步骤，不得覆盖历史步骤。

```
    """
    PLAN = "plan"
    POLICY = "policy"
    APPROVAL_REQUEST = "approval_request"
    TOOL_CALL = "tool_call"
    OBSERVATION = "observation"
    VERIFICATION = "verification"
    REPLAN = "replan"
    TERMINAL = "terminal"
class RiskLevel(enum.StrEnum):
    """工具风险级别（手册 3.5，4 个值，禁止增删改）。"""
    READ_ONLY = "read_only"
    WRITE = "write"
    DEVICE_COMMAND = "device_command"
    SENSITIVE = "sensitive"
# 终态集合（状态字符串），终态不得再次迁移
TERMINAL_STATUSES: frozenset[str] = frozenset(
    {s.value for s in (AgentStatus.SUCCEEDED, AgentStatus.FAILED, AgentStatus.CANCELLED, AgentStatu…
)
# 合法状态迁移表（确定性内核的状态机，见 runtime.py::_transition）
LEGAL_TRANSITIONS: dict[str, set[str]] = {
    AgentStatus.CREATED.value: {AgentStatus.PLANNING.value},
    AgentStatus.PLANNING.value: {AgentStatus.WAITING_POLICY.value, AgentStatus.EXECUTING.value},
    AgentStatus.WAITING_POLICY.value: {
        AgentStatus.WAITING_APPROVAL.value,
        AgentStatus.EXECUTING.value,
    },
    AgentStatus.WAITING_APPROVAL.value: {AgentStatus.EXECUTING.value},
    AgentStatus.EXECUTING.value: {AgentStatus.OBSERVING.value, AgentStatus.VERIFYING.value},
    AgentStatus.OBSERVING.value: {AgentStatus.EXECUTING.value, AgentStatus.VERIFYING.value},
    AgentStatus.VERIFYING.value: {AgentStatus.PLANNING.value},  # 观察→验证→重规划→安全终止
}
# ---------------------------------------------------------------------
# 请求 / 运行 / 步骤 / 结果
# ---------------------------------------------------------------------
@dataclass
class AgentRunRequest:
    """触发一次 Agent 运行的请求。
    - trigger_type：事件/人工/定时触发来源标识
    - objective：本次运行目标（业务目标描述）
    - actor / role：发起人与其角色（策略守卫按角色放行工具）
    - params：业务参数，供规则计划绑定（如 {"event_id": "evt_001"}）
    - idempotency_key：重复触发幂等键（同一键复用已有 run，不新建）
    - plan_key：选择哪一套规则计划（对应 RuntimeConfig.rule_plans 的键）。
      None 表示用 RuntimeConfig.rule_plan（即既有单角色派单管道）。
      多角色协同只是「换一套计划」，不改内核状态机，也不改
      任何一步的既有语义。
    - lessons_context：调用方提供的业务上下文，供跨 run 经验按条件检索
      （如 {"merge_candidate": true, "high_priority": true}）。
```

```
    内核不自己从数据库里翻业务状态：调用方知道什么就传什么，
    不知道就不传：这样经验引用永远是基于真实已知事实的，
    而不是内核猜出来的。
    """
    trigger_type: str
    objective: str
    actor: str = "operator"
    role: str = "operator"
    params: dict[str, Any] = field(default_factory=dict)
    idempotency_key: str | None = None
    trace_id: str | None = None
    plan_key: str | None = None
    lessons_context: dict[str, Any] = field(default_factory=dict)
@dataclass
class AgentStep:
    """运行轨迹步骤（字段对齐 t_agent_step）。
    解密预算：只保存决策摘要（decision_summary）与输入/输出哈希
     （input_hash / output_hash），绝不保存模型私有思维链或原始载荷。
    """
    step_id: str
    run_id: str
    step_no: int
    step_type: str  # AgentStepType 的值
    decision_summary: str
    tool_name: str | None = None
    tool_version: str | None = None
    input_hash: str | None = None
    output_hash: str | None = None
    status: str = "ok"  # ok / failed / pending
    latency_ms: int = 0
    error_code: str | None = None
    created_at: datetime | None = None
@dataclass
class AgentRun:
    """一次运行的聚合根（字段对齐 t_agent_run）。
    `request` 与 `runtime_state` 是内存实现的续跑上下文：
    审批挂起后 resume() 需要知道「计划执行到哪、绑定变量是什么」。
    WP-02 落 ORM 时这两块应序列化为 run_state JSON 字段（见风险清单）。
    """
    run_id: str
    trigger_type: str
    objective: str
    status: str  # AgentStatus 的值
    request: AgentRunRequest = field(repr=False)
    policy_version: str | None = None
    started_at: datetime | None = None
    finished_at: datetime | None = None
    termination_reason: str | None = None
    trace_id: str | None = None
    created_at: datetime | None = None
```

```
    updated_at: datetime | None = None



    runtime_state: dict[str, Any] = field(default_factory=dict, repr=False)
@dataclass
class AgentRunResult:
    """run / cancel / resume 的返回结果。"""
    run_id: str
    status: str  # AgentStatus 的值
    termination_reason: str | None = None
    error_code: str | None = None
    steps: list[AgentStep] = field(default_factory=list)
    trace_id: str | None = None
    objective: str = ""
    trigger_type: str = ""
    policy_version: str | None = None
    finished_at: datetime | None = None
    pending_approval_ids: list[str] = field(default_factory=list)
    idempotent_replay: bool = False
@dataclass
class RuntimeStatus:
    """`AgentRuntime.status()` 的返回（运行期快照）。
    - state: idle（无活跃 run） / running（存在活跃 run）
    - model_available: 大模型是否可用
    - rule_mode: 当前是否运行在确定性规则模式
    """
    state: str
    model_available: bool
    rule_mode: bool
    policy_version: str
    active_runs: int
    total_runs: int
    pending_approvals: int
    tools_registered: int
    uptime_ms: int
# ---------------------------------------------------------------------
# 规则计划（确定性规划，无大模型）
# ---------------------------------------------------------------------
@dataclass(frozen=True)
class Expectation:
    """步骤期望（确定性验证规则，代替模型判断）。
    - key: 检查输出 data 的哪个字段
    - min_items: 字段为列表/字符串时最小长度
    - contains: 字段必须等于的候选值集合（命中任一即通过）
    - on_violation: terminate（安全失败） / replan（重规划重试）
    - error_code: 不满足时的结构化错误码
    """
    key: str
    min_items: int | None = None
    contains: tuple[Any, ...] = ()
    on_violation: str = "terminate"
    error_code: str = "tool_failed"
```

```
@dataclass(frozen=True)



class RuleStep:
    """规则计划中的一个步骤。
    - input：支持 "{var}" 占位符，由已完成的工具输出与请求参数绑定
    - on_error：工具执行失败（tool_timeout / tool_failed）时 terminate / replan
    - role：产出该步骤的智能体角色（多角色协同模式用，单角色模式为 None）。
      这是内核内的计划元数据，不落 t_agent_step：轨迹的角色归属由
      `AgentRun.runtime_state["step_roles"]`（step_no → 角色）承载，
      和 plan/bindings 一样随 run 状态快照走。这样既能让轨迹与前端显示
      「这一步是谁做的」，又不改动手册 3.7 冻结的步骤表结构。
    """
    tool: str
    input: dict[str, Any]
    summary: str
    expect: Expectation | None = None
    on_error: str = "terminate"
    role: str | None = None
# ---------------------------------------------------------------------
# 运行时配置（可注入，规则模式的全部可调参数）
# ---------------------------------------------------------------------
@dataclass
class RuntimeConfig:
    """运行时配置（手册 3.6 可注入依赖之一）。
    `model_available=False` 表示模型不可用：内核自动运行在
    确定性规则模式，核心派单闭环不降级。
    """
    policy_version: str = "rules-v1.0"
    max_steps: int = 30          # 步骤轨迹上限（安全终止，防无限循环）
    max_replans: int = 3         # 重规划上限（观察→验证→重规划→安全终止）
    tool_timeout_ms: int = 5000  # 工具超时（毫秒）
    retry_limit: int = 1         # 工具失败/超时后的重试上限
    approval_timeout_ms: int = 300_000  # 审批超时（毫秒），超时 → approval_timeout
    run_ttl_ms: int = 86_400_000        # 运行总时长上限，超时 → run_expired
    model_available: bool = False       # 模型不可用 → 规则模式
    rule_plan: list[dict[str, Any]] | None = None  # None → 内置派单管道
    # 可选计划注册表：request.plan_key 命中时用它，未命中回落 rule_plan。
    # 多角色协同（事件研判 Agent → 调度执行 Agent）即由此挂载，
    # 保证单角色默认路径逐字节不变。
    rule_plans: dict[str, list[dict[str, Any]]] | None = None
    # 跨 run 经验闭环（lessons）：开启后规划阶段会检索同类事件的历史经验，
    # 命中则写进 plan 步骤摘要；run 到终态时按确定性规则复盘提炼新经验。
    # 默认关闭：评测基线场景自带 RuntimeConfig，行为必须逐字节不变。
    lessons_enabled: bool = False
    require_approval_risk: tuple[RiskLevel, ...] = (
        RiskLevel.DEVICE_COMMAND,
        RiskLevel.SENSITIVE,
    )
# ===== backend/app/services/agents/repositories.py =====
"""运行与审批仓储：接口（可注入）+ 内存实现。
唯一约束（手册 3.7）在内存实现中强制执行：
```

```
    - run_id / step_id / approval_id 唯一



    - t_agent_step 的 (run_id, step_no) 唯一
    - 幂等键 → run_id 索引唯一 (重复触发复用 run)
字段名与 WP-02 ORM 对齐: ApprovalRecord 对齐 t_agent_approval,
AgentRun / AgentStep 对齐 t_agent_run / t_agent_step。
"""
from __future__ import annotations
from dataclasses import dataclass
from datetime import datetime, timezone
from typing import Any, Protocol
from app.services.agents.errors import AgentNotFoundError, TaskConflictError
from app.services.agents.model import AgentRun, AgentStep
def _time_key(value: datetime | None) -> datetime:
    """统一排序时间: None 视为最早, naive 按 UTC, aware 转 UTC。"""
    if value is None:
        return datetime.min.replace(tzinfo=timezone.utc)
    if value.tzinfo is None:
        return value.replace(tzinfo=timezone.utc)
    return value.astimezone(timezone.utc)
@dataclass
class ApprovalRecord:
    """一条人工审批记录 (字段对齐 t_agent_approval) 。
    decision: None=待审批 / "approved"=通过 / "rejected"=拒绝 /
    "cancelled"=随 run 取消。
    """
    approval_id: str
    run_id: str
    requested_action: str
    risk_level: str  # RiskLevel 的值
    requested_by: str
    decided_by: str | None = None
    decision: str | None = None
    reason: str | None = None
    requested_at: datetime | None = None
    decided_at: datetime | None = None
# ---------------------------------------------------------------------
# Run 仓储
# ---------------------------------------------------------------------
class RunRepository(Protocol):
    """运行仓储协议 (可注入, 见手册 3.6) 。
    协议方法体做参数校验后抛 NotImplementedError: 既保证形参被真实
    使用 (静态守卫) , 又保持纯协议语义: 具体行为由注入的实现提供。
    """
    def create(self, run: AgentRun) -> None:
        if not isinstance(run, AgentRun):
            raise TypeError(f"run 必须是 AgentRun: {type(run).__name__}")
        raise NotImplementedError("RunRepository.create 未实现")
    def save(self, run: AgentRun) -> None:
        if not isinstance(run, AgentRun):
            raise TypeError(f"run 必须是 AgentRun: {type(run).__name__}")
```

```
        raise NotImplementedError("RunRepository.save 未实现")


    def get(self, run_id: str) -> AgentRun | None:
        if not isinstance(run_id, str):
            raise TypeError(f"run_id 必须是 str：{type(run_id).__name__}")
        raise NotImplementedError("RunRepository.get 未实现")
    def list(self, *, status: str | None = None, limit: int | None = None) -> list[AgentRun]:
        if status is not None and not isinstance(status, str):
            raise TypeError(f"status 必须是 str 或 None：{type(status).__name__}")
        if limit is not None and (not isinstance(limit, int) or isinstance(limit, bool) or limit < …
            raise TypeError(f"limit 必须是非负整数或 None：{limit!r}")
        raise NotImplementedError("RunRepository.list 未实现")
    def find_by_idempotency_key(self, key: str) -> AgentRun | None:
        if not isinstance(key, str) or not key:
            raise TypeError(f"key 必须是非空 str：{key!r}")
        raise NotImplementedError("RunRepository.find_by_idempotency_key 未实现")
    def add_step(self, step: AgentStep) -> None:
        if not isinstance(step, AgentStep):
            raise TypeError(f"step 必须是 AgentStep：{type(step).__name__}")
        raise NotImplementedError("RunRepository.add_step 未实现")
    def steps(self, run_id: str) -> list[AgentStep]:
        if not isinstance(run_id, str):
            raise TypeError(f"run_id 必须是 str：{type(run_id).__name__}")
        raise NotImplementedError("RunRepository.steps 未实现")
    def count(self) -> int:
        raise NotImplementedError("RunRepository.count 未实现")
class InMemoryRunRepository:
    """内存运行仓储。"""
    def __init__(self) -> None:
        self._runs: dict[str, AgentRun] = {}
        self._steps: dict[str, list[AgentStep]] = {}
        self._step_keys: set[tuple[str, int]] = set()
        self._step_ids: set[str] = set()
        self._by_idem: dict[str, str] = {}
    def create(self, run: AgentRun) -> None:
        if run.run_id in self._runs:
            raise TaskConflictError(f"run_id 重复：{run.run_id}")
        self._runs[run.run_id] = run
        key = run.request.idempotency_key if run.request is not None else None
        if key:
            if key in self._by_idem:
                raise TaskConflictError(f"幂等键重复：{key}")
            self._by_idem[key] = run.run_id
    def save(self, run: AgentRun) -> None:
        if run.run_id not in self._runs:
            raise AgentNotFoundError(f"run 不存在：{run.run_id}")
        self._runs[run.run_id] = run
    def get(self, run_id: str) -> AgentRun | None:
        return self._runs.get(run_id)
    def list(self, *, status: str | None = None, limit: int | None = None) -> list[AgentRun]:
        runs = sorted(self._runs.values(), key=lambda r: _time_key(r.created_at))
```

```
        if status is not None:
            runs = [r for r in runs if r.status == status]
        if limit is not None:
            runs = runs[:limit]
        return runs
    def find_by_idempotency_key(self, key: str) -> AgentRun | None:
        run_id = self._by_idem.get(key)
        if run_id is None:
            return None
        return self._runs.get(run_id)
    def add_step(self, step: AgentStep) -> None:
        if step.step_id in self._step_ids:
            raise TaskConflictError(f"step_id 重复：{step.step_id}")
        if (step.run_id, step.step_no) in self._step_keys:
            raise TaskConflictError(f"(run_id, step_no) 重复：{step.run_id} #{step.step_no}")
        self._steps.setdefault(step.run_id, []).append(step)
        self._step_keys.add((step.run_id, step.step_no))
        self._step_ids.add(step.step_id)
    def steps(self, run_id: str) -> list[AgentStep]:
        steps = sorted(self._steps.get(run_id, []), key=lambda s: s.step_no)
        return list(steps)
    def count(self) -> int:
        return len(self._runs)
# ---------------------------------------------------------------------
# 审批仓储
# ---------------------------------------------------------------------
class ApprovalRepository(Protocol):
    """审批仓储协议（可注入，见手册 3.6）。
    协议方法体做参数校验后抛 NotImplementedError：既保证形参被真实
    使用（静态守卫），又保持纯协议语义：具体行为由注入的实现提供。
    """
    def create(self, record: ApprovalRecord) -> ApprovalRecord:
        if not isinstance(record, ApprovalRecord):
            raise TypeError(f"record 必须是 ApprovalRecord：{type(record).__name__}")
        raise NotImplementedError("ApprovalRepository.create 未实现")
    def get(self, approval_id: str) -> ApprovalRecord | None:
        if not isinstance(approval_id, str):
            raise TypeError(f"approval_id 必须是 str：{type(approval_id).__name__}")
        raise NotImplementedError("ApprovalRepository.get 未实现")
    def list_pending(self, run_id: str | None = None) -> list[ApprovalRecord]:
        if run_id is not None and not isinstance(run_id, str):
            raise TypeError(f"run_id 必须是 str 或 None：{type(run_id).__name__}")
        raise NotImplementedError("ApprovalRepository.list_pending 未实现")
    def list_all(self) -> list[ApprovalRecord]:
        """列出全部审批记录（含已决策），按请求时间升序。"""
        raise NotImplementedError("ApprovalRepository.list_all 未实现")
    def decide(
        self,
        approval_id: str,
        decided_by: str,
```

```
        decision: str,
        reason: str | None = None,
    ) -> ApprovalRecord:
        if not isinstance(approval_id, str):
            raise TypeError(f"approval_id 必须是 str：{type(approval_id).__name__}")
        if not isinstance(decided_by, str):
            raise TypeError(f"decided_by 必须是 str：{type(decided_by).__name__}")
        if decision not in ("approved", "rejected", "cancelled"):
            raise ValueError(f"非法决策：{decision!r}")
        if reason is not None and not isinstance(reason, str):
            raise TypeError(f"reason 必须是 str 或 None：{type(reason).__name__}")
        raise NotImplementedError("ApprovalRepository.decide 未实现")
    def count(self) -> int:
        raise NotImplementedError("ApprovalRepository.count 未实现")
class InMemoryApprovalRepository:
    """内存审批仓储。"""
    def __init__(self) -> None:
        self._records: dict[str, ApprovalRecord] = {}
    def create(self, record: ApprovalRecord) -> ApprovalRecord:
        if record.approval_id in self._records:
            raise TaskConflictError(f"approval_id 重复：{record.approval_id}")
        self._records[record.approval_id] = record
        return record
    def get(self, approval_id: str) -> ApprovalRecord | None:
        return self._records.get(approval_id)
    def list_pending(self, run_id: str | None = None) -> list[ApprovalRecord]:
        records = [r for r in self._records.values() if r.decision is None]
        if run_id is not None:
            records = [r for r in records if r.run_id == run_id]
        records.sort(key=lambda r: _time_key(r.requested_at))
        return records
    def list_all(self) -> list[ApprovalRecord]:
        records = sorted(
            self._records.values(),
            key=lambda r: _time_key(r.requested_at),
        )
        return list(records)
    def decide(
        self,
        approval_id: str,
        decided_by: str,
        decision: str,
        reason: str | None = None,
    ) -> ApprovalRecord:
        record = self._records.get(approval_id)
        if record is None:
            raise AgentNotFoundError(f"审批不存在：{approval_id}")
        if record.decision is not None and record.decision != "cancelled":
            # 已决策的审批不允许被覆盖（cancelled 允许被覆盖为正式决策）
            raise TaskConflictError(f"审批 {approval_id} 已决策：{record.decision}")
```

```
        if decision not in ("approved", "rejected", "cancelled"):
            raise AgentNotFoundError(f"非法审批决策：{decision}")
        record.decision = decision
        record.decided_by = decided_by
        record.reason = reason
        record.decided_at = datetime.now(timezone.utc)
        return record
    def count(self) -> int:
        return len(self._records)
# ===== backend/app/services/agents/factory.py =====
"""仓储工厂（总控集成）。
WP-10「可选仓储工厂」：无数据库引擎时返回内存仓储（默认行为，
离线测试不依赖数据库、不破坏确定性）；提供 SQLAlchemy engine 时返回
WP-10 持久化仓储（SqlAlchemyRunRepository / SqlAlchemyApprovalRepository）。
设计约束：
- 默认（engine=None）必须等价于直接使用内存仓储，零配置可用。
- 数据库不可用时不得破坏离线测试：由上层选择是否提供 engine。
- 工厂只做选择与装配，不改变任何仓储语义（冻结协议见 repositories.py）。
- 仓储自身管理 ID/时钟（内存仓储无需注入；SQLAlchemy 仓储用 DB 序列）。
"""
from __future__ import annotations
from sqlalchemy.engine import Engine
from app.services.agents.persistent_repository import (
    SqlAlchemyApprovalRepository,
    SqlAlchemyRunRepository,
)
from app.services.agents.repositories import (
    InMemoryApprovalRepository,
    InMemoryRunRepository,
)
def create_run_repository(engine: Engine | None = None):
    """创建运行仓储。
    engine=None → InMemoryRunRepository（默认）；
    engine 提供 → SqlAlchemyRunRepository（持久化，乐观锁/幂等由仓储保证）。
    """
    if engine is None:
        return InMemoryRunRepository()
    return SqlAlchemyRunRepository(engine=engine)
def create_approval_repository(engine: Engine | None = None):
    """创建审批仓储。
    engine=None → InMemoryApprovalRepository（默认）；
    engine 提供 → SqlAlchemyApprovalRepository（持久化）。
    """
    if engine is None:
        return InMemoryApprovalRepository()
    return SqlAlchemyApprovalRepository(engine=engine)
def create_repositories(engine: Engine | None = None) -> tuple[object, object]:
    """成对创建 run + approval 仓储，返回 (run_repository, approval_repository)。
    供 AgentRuntime / API 装配点使用：数据库可选，默认内存。
    """
```

```
    return (
        create_run_repository(engine),
        create_approval_repository(engine),
    )
# ===== backend/app/services/agents/errors.py =====
"""Agent 内核结构化错误。
错误码清单与 Harness 执行手册 3.3 完全一致（12 个，禁止增删改），
由 `backend/tests/test_agent_runtime.py` 中的「错误码枚举对账守卫」锁定。
约定：
- `AgentError` 是所有内核异常的基类。
- `code` 取冻结错误码字符串；`code=None` 表示调用侧错误
  （如 run 不存在），这类错误不会被写入运行轨迹，也不会成为
  run 的 `termination_reason`。
- 运行结果中的 `termination_reason` / 步骤 `error_code` 只允许出现
  冻结错误码或状态值本身（succeeded / cancelled），由对账守卫测试校验。
"""
from __future__ import annotations
from typing import Final
# ---------------------------------------------------------------------
# 冻结错误码（手册 3.3）—— 与测试中的对账守卫同源
# ---------------------------------------------------------------------
ERROR_CODES: Final[frozenset[str]] = frozenset(
    {
        "no_robot_available",
        "tool_timeout",
        "tool_failed",
        "policy_denied",
        "approval_rejected",
        "approval_timeout",
        "task_conflict",
        "invalid_tool_input",
        "invalid_tool_output",
        "max_steps_exceeded",
        "run_expired",
        "internal_error",
    }
)
class ErrorCode:
    """结构化错误码常量（与 ERROR_CODES 一一对应）。"""
    NO_ROBOT_AVAILABLE = "no_robot_available"
    TOOL_TIMEOUT = "tool_timeout"
    TOOL_FAILED = "tool_failed"
    POLICY_DENIED = "policy_denied"
    APPROVAL_REJECTED = "approval_rejected"
    APPROVAL_TIMEOUT = "approval_timeout"
    TASK_CONFLICT = "task_conflict"
    INVALID_TOOL_INPUT = "invalid_tool_input"
    INVALID_TOOL_OUTPUT = "invalid_tool_output"
    MAX_STEPS_EXCEEDED = "max_steps_exceeded"
    RUN_EXPIRED = "run_expired"
```

```
    INTERNAL_ERROR = "internal_error"



def is_error_code(value: str | None) -> bool:
    """判断字符串是否为冻结错误码。"""
    return value in ERROR_CODES
class AgentError(Exception):
    """Agent 内核异常基类。
    `code` 必须在 `ERROR_CODES` 内；`None` 表示调用侧错误
    （不会进入运行轨迹）。
    """
    def __init__(self, message: str, *, code: str | None = None) -> None:
        self.code = code
        super().__init__(message)
class AgentNotFoundError(AgentError):
    """run / 审批 / 记忆对象不存在（调用侧错误，无冻结错误码）。"""
    def __init__(self, message: str) -> None:
        super().__init__(message, code=None)
class TaskConflictError(AgentError):
    """唯一约束冲突 / 幂等键冲突（run_id、step_id、(run_id, step_no)、
    memory_id、approval_id、幂等键）。"""
    def __init__(self, message: str) -> None:
        super().__init__(message, code=ErrorCode.TASK_CONFLICT)
class ToolTimeoutError(AgentError):
    def __init__(self, message: str) -> None:
        super().__init__(message, code=ErrorCode.TOOL_TIMEOUT)
class ToolFailedError(AgentError):
    def __init__(self, message: str) -> None:
        super().__init__(message, code=ErrorCode.TOOL_FAILED)
class PolicyDeniedError(AgentError):
    def __init__(self, message: str) -> None:
        super().__init__(message, code=ErrorCode.POLICY_DENIED)
class ApprovalRejectedError(AgentError):
    def __init__(self, message: str) -> None:
        super().__init__(message, code=ErrorCode.APPROVAL_REJECTED)
class ApprovalTimeoutError(AgentError):
    def __init__(self, message: str) -> None:
        super().__init__(message, code=ErrorCode.APPROVAL_TIMEOUT)
class InvalidToolInputError(AgentError):
    def __init__(self, message: str) -> None:
        super().__init__(message, code=ErrorCode.INVALID_TOOL_INPUT)
class InvalidToolOutputError(AgentError):
    def __init__(self, message: str) -> None:
        super().__init__(message, code=ErrorCode.INVALID_TOOL_OUTPUT)
class MaxStepsExceededError(AgentError):
    def __init__(self, message: str) -> None:
        super().__init__(message, code=ErrorCode.MAX_STEPS_EXCEEDED)
class RunExpiredError(AgentError):
    def __init__(self, message: str) -> None:
        super().__init__(message, code=ErrorCode.RUN_EXPIRED)
class InternalAgentError(AgentError):
    def __init__(self, message: str) -> None:
```

```
        super().__init__(message, code=ErrorCode.INTERNAL_ERROR)


# ===== backend/app/services/event.py =====
"""事件服务：接收上报 → 幂等判重 → 落库 → 投递派单队列 → 推送大屏。
这是平台数据流的入口。
"""
from __future__ import annotations
import json
from datetime import datetime
from typing import Any
from loguru import logger
from redis.asyncio import Redis
from sqlalchemy import func
from sqlalchemy.ext.asyncio import AsyncSession
from app.core.config import settings
from app.core.exceptions import AppException, ErrorCode
from app.core.geo import nearest_township
from app.models.event import Event, EventStatus, WasteClass
from app.repositories import DeviceRepository, EventRepository
from app.schemas import EventIngest, EventIngestResult
class EventService:
    """事件处理服务。"""
    def __init__(self, session: AsyncSession, redis: Redis | None = None) -> None:
        self.session = session
        self.redis = redis
        self.events = EventRepository(session)
        self.devices = DeviceRepository(session)
    async def ingest(
        self, payload: EventIngest, *, enqueue_dispatch: bool = True
    ) -> EventIngestResult:
        """接收一条事件上报。
        幂等保证：
            1. (device_id, seq) 唯一约束：数据库层面兜底
            2. 此处先查一次：快速返回，避免触发约束异常
        enqueue_dispatch=False 表示调用方会自己执行同步派单（HTTP/MQTT
        入口当前都是这个模式）：事件只落库，不再投 Redis Stream，避免
        同一条事件被同步派单与后台消费者重复处理。
        QoS 1 会重复投递，所以判重是必需的，不是可选优化。
        """
        # 幂等判重
        if await self.events.exists_by_device_seq(payload.device_id, payload.seq):
            logger.info(
                f"[事件] 重复上报 device={payload.device_id} seq={payload.seq}，忽略"
            )
            return EventIngestResult(
                accepted=True,
                event_id=payload.event_id,
                duplicate=True,
                message="重复上报（device_id+seq 已存在），已忽略",
            )
        if await self.events.get_by_event_id(payload.event_id) is not None:
```

```
            return EventIngestResult(
                accepted=True,
                event_id=payload.event_id,
                duplicate=True,
                message="重复上报（event_id 已存在），已忽略",
            )
        # 校验设备
        device = await self.devices.get_by_device_id(payload.device_id)
        if device is None:
            raise AppException(
                code=ErrorCode.DEVICE_NOT_FOUND,
                message=f"设备 {payload.device_id} 未注册",
            )
        # 落库
        event = await self.events.create(
            event_id=payload.event_id,
            device_id=payload.device_id,
            event_time=payload.timestamp,
            location=f"SRID=4326;POINT({payload.location.lng} {payload.location.lat})",
            main_class=payload.aggregate.main_class,
            det_count=payload.aggregate.count,
            max_confidence=payload.aggregate.max_confidence,
            evidence_url=payload.evidence_url,
            model_version=payload.model_version or settings.ai_model_version,
            seq=payload.seq,
            status=EventStatus.NEW,
            township=nearest_township(payload.location.lng, payload.location.lat),
        )
        # 更新设备心跳（事件本身就是活跃证明）
        device.last_heartbeat = datetime.now()
        if device.status != "online":
            device.status = "online"
        await self.session.flush()
        # 投递派单队列
        # 只有调用方没有同步派单时才入队；同步派单入口必须传 False，
        # 否则同一事件会被 dispatch_consumer 再派一次。
        if enqueue_dispatch:
            await self._enqueue_dispatch(event)
        logger.info(
            f"[事件] 入库 {event.event_id} 类别={WasteClass.LABELS.get(event.main_class, event.main…
            f"设备={event.device_id} 数量={event.det_count}"
        )
        return EventIngestResult(
            accepted=True,
            event_id=event.event_id,
            duplicate=False,
            message="事件已受理",
        )
    async def _enqueue_dispatch(self, event: Event) -> None:
        """把事件推入 Redis Stream，交给派单消费者处理（显式异步通道）。
```

```
        为什么用 Redis Streams 而不是:

        - 数据库轮询：延迟高、空转耗资源
        - 同步调用派单：机器人离线会阻塞 HTTP 请求
        - Kafka/RabbitMQ：对小团队是过度设计，Redis 本来就要用
        当前 HTTP/MQTT 两个入口都在提交后同步派单，调用 ingest 时
          必须传 enqueue_dispatch=False，不要在这里重复入队。本方法
          保留给「不需要同步 task_id 的异步入口」和批量补投使用。
        """
        if self.redis is None:
            return
        try:
            await self.redis.xadd(
                settings.redis_stream_events,
                {
                    "event_id": event.event_id,
                    "device_id": event.device_id,
                    "main_class": event.main_class,
                    "ts": datetime.now().isoformat(),
                },
                maxlen=10000,   # 防止 stream 无限增长
                approximate=True,
            )
        except Exception as exc:   # noqa: BLE001
            # 队列投递失败不应阻断事件入库，由定时补派兜底
            logger.error(f"[事件] 投递派单队列失败：{exc}")
    async def heatmap(
        self,
        *,
        hours: int = 24,
        grid_size: int = 500,
        main_class: str | None = None,
    ) -> list[dict[str, Any]]:
        """热力图聚合数据。"""
        return await self.events.heatmap_aggregate(
            hours=hours, grid_size=grid_size, main_class=main_class
        )
    async def event_lnglat(self, event: Event) -> tuple[float | None, float | None]:
        """读取事件经纬度。"""
        from sqlalchemy import select
        stmt = select(func.ST_X(event.location), func.ST_Y(event.location))
        row = (await self.session.execute(stmt)).first()
        if row is None:
            return None, None
        return (
            float(row[0]) if row[0] is not None else None,
            float(row[1]) if row[1] is not None else None,
        )
# ===== backend/app/services/agents/lessons.py =====
"""跨 run 经验（lessons）——让确定性规则模式也具备「可进化」闭环。
## 这个模块解决什么问题
```

规则模式的智能体有一个天然短板：它每次运行都从零开始，上一单踩过的坑

下一单还会再踩。头部项目（Mem0 / Letta）用向量记忆解决这件事，代价是
要引入 embedding 模型、向量库和一整套不确定的召回链路：本项目的纪律
是不让核心能力依赖模型，所以这里走的是一条确定性经验回路：

    复盘（run 到终态）
      → 用可复现的规则从真实终态里提炼经验（lesson）
      → 落到 MemoryStore（memory_type=episodic，scope=事件类别）
      → 下一次 run 规划时按「条件」检索命中
      → 命中结果写进 plan 步骤的决策摘要，并按次递增命中计数
      → 轨迹里因此能看到「本次引用了哪条经验」

全程没有模型、没有向量、没有随机：同样的运行历史 + 同样的输入，
永远得到同样的经验引用，可离线复现。

为什么复用 MemoryStore 而不是新开一张表

手册 3.7 的 `t_agent_memory` 已经有
`memory_type / scope_type / scope_id / content / confidence / source_type`
五个字段，恰好够表达「哪一类事件（scope）在什么条件下该怎么做
（content）、有多可信（confidence）」。复用它 = 不改冻结契约。

命名空间约定（不新增列，靠取值区分）：

    memory_type = "episodic"：事件性经验（区别于 run 级 working 记忆）
    scope_type = "main_class"：经验按事件类别聚合
    scope_id   = <事件类别>：如 foam / plastic
    content   = {"kind":..., "condition":..., "guidance":...}
    source_id   = 提炼出该经验的 run_id

诚实性约束（这个项目的老规矩）

1. 经验只由确定性规则从 run 的真实终态提炼：失败就是失败，
   成功就是成功，不做「推测性归因」。
2. 置信度不是拍脑袋的常数：初始值由经验种类的证据强度决定，
   之后随命中次数单调上调并封顶，永远不到 1.0（规则模式的经验是启发式，
   不是定理）。
3. 存储是进程内的（MemoryStore 目前只有内存实现），只在这一个后端
   进程活着期间累积。重启后经验清零：这一点写在这里，也写在
   `agents.py` 的装配注释里，不假装它持久化了。
"""
```
from __future__ import annotations
from dataclasses import dataclass
from datetime import datetime, timezone
from typing import Any
from app.services.agents.memory import MemoryEntry, MemoryStore
# 记忆命名空间（见模块 docstring）
LESSON_MEMORY_TYPE = "episodic"
LESSON_SCOPE_TYPE = "main_class"
LESSON_SOURCE_TYPE = "run_review"
# 经验种类：取值即语义，不允许自由字符串，便于穷举与对账。
LESSON_KINDS: tuple[str, ...] = (
    "merge_preferred",        # 近邻已有同类活跃工单 → 合并优于新建
    "replan_recovered",       # 首次工具失败后重规划可恢复 → 允许一次重试
    "high_risk_priority",     # 政策重点类别 → 优先派单、优先顺路机器人
    "no_robot_standby",       # 候选池为空/都不合格 → 先补池，别立刻重试
    "evidence_insufficient",  # 证据不足被研判拦下 → 转人工，别自动派单
)
```

```
# 每种经验的初始置信度（由该种类结论的证据强度决定，不是统一常数）


_KIND_BASE_CONFIDENCE: dict[str, float] = {
    # 合并是幂等语义保证的，证据最强
    "merge_preferred": 0.85,
    # 「重规划可恢复」来自一次真实恢复，样本少，起点低
    "replan_recovered": 0.60,
    # 类别优先级来自政策本身，不是统计结论
    "high_risk_priority": 0.80,
    # 资源不足是硬事实
    "no_robot_standby": 0.75,
    # 研判门禁是确定性规则
    "evidence_insufficient": 0.90,
}
# 单次命中带来的置信度增量，以及封顶（永不到 1.0）
_CONFIDENCE_STEP = 0.03
_CONFIDENCE_CEILING = 0.95
# 每种经验的「适用条件」谓词键（对应 lessons_context 里的字段）。
# 条件不是装饰：match() 会真的按它过滤，不满足就不引用。
_KIND_PREDICATES: dict[str, str] = {
    "merge_preferred": "merge_candidate",
    "no_robot_standby": "no_candidate",
    "replan_recovered": "",              # 无条件，凡同类事件都适用
    "high_risk_priority": "high_priority",
    "evidence_insufficient": "evidence_insufficient",
}
_KIND_TEXT: dict[str, tuple[str, str]] = {
    "merge_preferred": (
        "近邻已有同类活跃工单（合并窗口内）",
        "优先并入既有工单而非新建，减少重复出勤",
    ),
    "replan_recovered": (
        "同类事件出现过首次工具失败后重规划恢复",
        "首次调用失败时允许一次重规划，不要直接判死",
    ),
    "high_risk_priority": (
        "该类别属政策重点治理类别",
        "优先派单，并优先选择同类别顺路机器人",
    ),
    "no_robot_standby": (
        "同类事件曾因候选机器人全部不合格而无法派单",
        "先补池/等待充电与清仓，不要立刻重复触发自动派单",
    ),
    "evidence_insufficient": (
        "同类事件曾因证据不足被研判拦下",
        "缺证据图或置信度不达标时转人工复核，不做自动派单",
    ),
}
# 命中次数上限（防止无限增长；也避免单条经验被"刷"到高置信）
_MAX_HIT_COUNT = 99
def _as_text(value: Any) -> str:
```

```
    return "" if value is None else str(value)



@dataclass
class Lesson:
    """一条经验（对外暴露的只读视图）。"""
    lesson_id: str
    scope_id: str            # 事件类别
    kind: str                # LESSON_KINDS 之一
    condition: str           # 人话条件
    guidance: str            # 结论 / 动作建议
    confidence: float
    hit_count: int = 0
    confirm_count: int = 0
    source_run_id: str | None = None
    created_at: datetime | None = None
    updated_at: datetime | None = None
    def describe(self) -> str:
        """给轨迹摘要用的一句话（控制在一条截图能读完的长度）。"""
        return f"{self.scope_id}/{self.kind}: {self.guidance} (置信度 {self.confidence:.2f}) "
@dataclass
class _LessonState:
    """内存中的当前态；每次变更都往 MemoryStore 追加一条审计记录。"""
    lesson: Lesson
    memory_id: str  # 最近一条记忆记录的 id（便于溯源）
class LessonStore:
    """经验库：在 MemoryStore 之上做确定性检索与命中计数。
    `memory` 注入既有的记忆仓储（内核的同一个 `self.memory`），
    所以「run 级 working 记忆」和「跨 run 经验」共用一套存储与脱敏纪律。
    `enabled=False` 时 `match()` 恒返回空列表、`harvest()` 直接返回——
    评测基线场景构建自己的 RuntimeConfig，不会因为本模块产生行为差异。
    """
    def __init__(
        self,
        memory: MemoryStore,
        *,
        enabled: bool = True,
        id_factory: Any = None,
        clock: Any = None,
    ) -> None:
        self.memory = memory
        self.enabled = enabled
        self._id_factory = id_factory
        self._clock = clock
        # key = (scope_id, kind)：同类同种经验只有一条，靠命中次数累积
        self._lessons: dict[tuple[str, str], _LessonState] = {}
    # ------------------------------------------------------------------
    # 查询
    # ------------------------------------------------------------------
    def all_lessons(self) -> list[Lesson]:
        """全部经验（按置信度降序、其次按创建时间）。"""
        items = [state.lesson for state in self._lessons.values()]
```

```
        items.sort(
            key=lambda item: (
                -item.confidence,
                item.created_at or datetime.min.replace(tzinfo=timezone.utc),
            )
        )
        return items
    def match(self, *, main_class: str, context: dict[str, Any] | None = None) -> list[Lesson]:
        """按事件类别 + 上下文条件检索命中的经验。
        条件判定完全确定性：谓词键在 `context` 里为真才命中。
        `replan_recovered` 无谓词，凡同类事件都适用。
        """
        if not self.enabled or not main_class:
            return []
        ctx = context or {}
        hits: list[Lesson] = []
        for (scope_id, kind), state in self._lessons.items():
            if scope_id != main_class:
                continue
            predicate = _KIND_PREDICATES.get(kind, "")
            if predicate and not ctx.get(predicate):
                continue
            hits.append(state.lesson)
        hits.sort(key=lambda item: (-item.confidence, item.kind))
        return hits
    # ------------------------------------------------------------------
    # 复盘 / 提炼
    # ------------------------------------------------------------------
    def harvest(
        self,
        *,
        run_id: str,
        main_class: str,
        outcome: str,
        termination_reason: str | None,
        bindings: dict[str, Any] | None,
        replan_count: int = 0,
    ) -> list[Lesson]:
        """从一个已到终态的 run 里提炼经验（确定性规则，不看模型）。
        返回本次新增/更新的经验列表，空列表表示这次复盘没有可提炼的结论
: 这是常态，不为了"有产出"而硬编一条经验出来。
        """
        if not self.enabled or not main_class:
            return []
        data = bindings or {}
        kinds: list[str] = []
        if outcome == "succeeded":
            if data.get("action") == "merged":
                kinds.append("merge_preferred")
            if int(replan_count or 0) > 0:
```

```
            kinds.append("replan_recovered")



        if data.get("risk_level") == "high":
            kinds.append("high_risk_priority")
    elif outcome == "failed":
        if termination_reason == "no_robot_available":
            kinds.append("no_robot_standby")
        if (
            termination_reason == "policy_denied"
            and data.get("recommended_action") == "manual_review"
        ):
            kinds.append("evidence_insufficient")
    return [
        self._upsert(scope_id=main_class, kind=kind, run_id=run_id)
        for kind in kinds
    ]
def note_hit(self, lessons: list[Lesson], *, run_id: str) -> None:
    """记录"这批经验被这一次 run 引用了"。
    与 harvest 的再次确认分开计数（hit_count / confirm_count）：
      「被引用」和「被新证据再次确认」是两件事，混在一个计数里
      会让"引用 5 次"变成一句无法解释的数字。
    """
    if not self.enabled:
        return
    for lesson in lessons:
        state = self._lessons.get((lesson.scope_id, lesson.kind))
        if state is None:
            continue
        self._bump(state, run_id=run_id, as_hit=True)
# ------------------------------------------------------------------
# 内部
# ------------------------------------------------------------------
def _upsert(self, *, scope_id: str, kind: str, run_id: str) -> Lesson:
    existing = self._lessons.get((scope_id, kind))
    if existing is not None:
        self._bump(existing, run_id=run_id, as_hit=False)
        return existing.lesson
    condition, guidance = _KIND_TEXT[kind]
    now = self._clock.now() if self._clock is not None else datetime.now(timezone.utc)
    lesson = Lesson(
        lesson_id=(
            self._id_factory.new("lsn_")
            if self._id_factory is not None
            else f"lsn_{scope_id}_{kind}"
        ),
        scope_id=scope_id,
        kind=kind,
        condition=condition,
        guidance=guidance,
        confidence=_KIND_BASE_CONFIDENCE[kind],
        hit_count=0,
```

```
            confirm_count=0,
            source_run_id=run_id,
            created_at=now,
            updated_at=now,
        )
        memory_id = self._append_memory(lesson, run_id=run_id)
        self._lessons[(scope_id, kind)] = _LessonState(lesson=lesson, memory_id=memory_id)
        return lesson
    def _bump(self, state: _LessonState, *, run_id: str, as_hit: bool) -> None:
        lesson = state.lesson
        if as_hit:
            lesson.hit_count = min(_MAX_HIT_COUNT, lesson.hit_count + 1)
        else:
            lesson.confirm_count = min(_MAX_HIT_COUNT, lesson.confirm_count + 1)
        # 置信度只由"证据累积"驱动（引用 + 确认），两种证据等价计入
        evidence = lesson.hit_count + lesson.confirm_count
        lesson.confidence = min(
            _CONFIDENCE_CEILING,
            round(_KIND_BASE_CONFIDENCE[lesson.kind] + _CONFIDENCE_STEP * evidence, 4),
        )
        lesson.updated_at = (
            self._clock.now() if self._clock is not None else datetime.now(timezone.utc)
        )
        lesson.source_run_id = run_id
        state.memory_id = self._append_memory(lesson, run_id=run_id)
    def _append_memory(self, lesson: Lesson, *, run_id: str) -> str:
        """把经验当前态追加进 MemoryStore（只追加，不就地改写历史）。
        为什么是追加而不是更新：记忆表的语义是「留痕」。就地改写会让
        "这条经验是怎么变可信的" 无从追溯；追加之后，命中计数与置信度的
        演化本身就是一条可回放的审计线。
        """
        entry = MemoryEntry(
            memory_id="",
            memory_type=LESSON_MEMORY_TYPE,
            scope_type=LESSON_SCOPE_TYPE,
            scope_id=lesson.scope_id,
            content={
                "kind": lesson.kind,
                "condition": lesson.condition,
                "guidance": lesson.guidance,
                "hit_count": lesson.hit_count,
                "confirm_count": lesson.confirm_count,
            },
            confidence=lesson.confidence,
            source_type=LESSON_SOURCE_TYPE,
            source_id=run_id,
            created_at=lesson.updated_at,
        )
        saved = self.memory.save(entry)
        return saved.memory_id
```

```
    def __len__(self) -> int:
        return len(self._lessons)
def summarize_hits(lessons: list[Lesson]) -> str:
    """把命中经验拼成轨迹摘要里的「本次引用经验」片段。
    只列最强的一条 + 总条数：plan 步骤的决策摘要要能在一屏里读完，
    把 5 条经验全文贴进去等于没人看。
    """
    if not lessons:
        return ""
    top = lessons[0]
    if len(lessons) == 1:
        return f"本次引用经验：{top.describe()}"
    return f"本次引用经验 {len(lessons)} 条，首条：{top.describe()}"
__all__ = [
    "LESSON_KINDS",
    "LESSON_MEMORY_TYPE",
    "LESSON_SCOPE_TYPE",
    "LESSON_SOURCE_TYPE",
    "Lesson",
    "LessonStore",
    "summarize_hits",
]
# ===== backend/app/services/consumer.py =====
"""后台消费者与定时任务。
两个常驻 asyncio task：
1. dispatch_consumer：消费 Redis Streams 中的事件，执行派单
2. pending_dispatcher：定时补派（处理机器人离线期间积压的事件）
为什么用 Redis Streams 而不是：
- 数据库轮询：延迟高、空转耗资源
- 同步调用派单：机器人离线会阻塞 HTTP 请求
- Kafka/RabbitMQ：对小团队是过度设计，Redis 本来就要用
Streams 相比 pub/sub 的优势：消费者组 + ACK + pending list，消息不会丢。
"""
from __future__ import annotations
import asyncio
from datetime import datetime
from loguru import logger
from app.core.config import settings
from app.core.deps import get_redis
from app.db.session import get_session_factory
# PEL 回收的最小空闲时间：消息挂起超过这么久才认领回来重试。
# 不能设 0：否则会把「刚投递、正在处理」的消息也抢回来重复处理。
PEL_MIN_IDLE_MS = 60_000
def _stream_feature_unsupported(exc: BaseException) -> bool:
    """判断 Redis 是否缺少 Streams 消费组/认领能力。
    开发机上可能误连旧版 Redis（没有 XGROUP/XAUTOCLAIM）。这不是业务
    处理失败，不能让后台任务每 3 秒刷一次同样的错误；应明确降级到
    数据库补派（pending_dispatcher）。
    """
    message = str(exc).lower()
```

```
    return "unknown command" in message and (
        "xgroup" in message or "xautoclaim" in message or "xreadgroup" in message
    )
async def _reclaim_pending(
    redis,
    stream: str,
    group: str,
    consumer_name: str,
    min_idle_ms: int = PEL_MIN_IDLE_MS,
) -> list[tuple[str, dict]]:
    """认领 PEL (Pending Entries List) 里挂起过久的消息，返回待处理条目。
    为什么必须有这一段：这是 Redis Streams 最经典的坑：
        `xreadgroup(streams={stream: ">"})` 只读新消息，
        永远不返回 PEL 里的条目。而 `_process_entry` 在「无可用机器人」时
        刻意不 ACK，把消息留在 PEL 等后续重试。两者一叠加：
          - 这些消息永远不会被重投（哪怕机器人后来上线了）；
          - PEL 只增不减，且不受 stream 的 maxlen 裁剪影响；
          - 功能上被定时补派从数据库兜住了，所以从外面完全看不出来。
        于是「留着重试」实际上变成了「永久滞留 + 内存缓慢泄漏」。
    认领回来的条目会走一遍 `_process_entry`：
      - 事件仍未派单且有机器人了 → 正常派单并 ACK；
      - 事件已被定时补派处理掉 → 状态不是 NEW，直接 ACK，PEL 随之清空。
    换言之，这一段同时解决了「重试」和「PEL 清理」两件事。
    """
    # redis-py 8.x 签名：xautoclaim(name, groupname, consumername, min_idle_time, ...)
    result = await redis.xautoclaim(
        stream, group, consumer_name, min_idle_ms, count=10
    )
    # 返回 [next_start_id, entries, deleted_ids]；只要中间那段条目
    if not isinstance(result, (list, tuple)) or len(result) < 2:
        return []
    entries = result[1] or []
    return [(entry_id, fields) for entry_id, fields in entries]
async def dispatch_consumer() -> None:
    """派单消费者：从 Redis Stream 读取事件并执行派单。
    使用消费者组（consumer group）保证消息不丢；
    处理失败进入死信 stream 供人工排查。
    """
    from app.core.exceptions import NoRobotAvailableError
    from app.models.event import EventStatus
    from app.models.task import TaskStatus
    from app.repositories import EventRepository, TaskRepository
    from app.services.dispatch import DispatchEngine, finalize_dispatch
    stream = settings.redis_stream_events
    group = settings.redis_consumer_group
    consumer_name = f"worker-{datetime.now():%H%M%S}"
    # 等待 Redis 就绪
    redis = None
    for _ in range(10):
        try:
```

```
            redis = await get_redis()
            await redis.ping()
            break
        except Exception:   # noqa: BLE001
            await asyncio.sleep(2)
    if redis is None:
        logger.error("[消费者] Redis 不可用，派单消费者退出")
        return
    # 创建消费者组（幂等）
    try:
        await redis.xgroup_create(stream, group, id="0", mkstream=True)
        logger.info(f"[消费者] 创建消费者组 {group} on {stream}")
    except Exception as exc:   # noqa: BLE001
        if _stream_feature_unsupported(exc):
            logger.warning(
                "[消费者] 当前 Redis 不支持消费组/XAUTOCLAIM，"
                "已切换为数据库定时补派；Redis Streams 消费未启用"
            )
            return
        if "BUSYGROUP" not in str(exc):
            logger.warning(f"[消费者] 创建消费者组失败：{exc}")
    logger.info(f"[消费者] 派单消费者已启动：{consumer_name}")
    while True:
        try:
            # 先回收挂起过久的消息（PEL）
            # 放在读新消息之前：这些条目已经等了一轮，优先级更高。
            for entry_id, fields in await _reclaim_pending(
                redis, stream, group, consumer_name
            ):
                await _process_entry(
                    redis, entry_id, fields, stream, group,
                    DispatchEngine, EventRepository, EventStatus,
                )
            # 再阻塞读取新消息（block=5000ms 便于响应取消）
            messages = await redis.xreadgroup(
                groupname=group,
                consumername=consumer_name,
                streams={stream: ">"},
                count=10,
                block=5000,
            )
            if not messages:
                continue
            for _stream_name, entries in messages:
                for entry_id, fields in entries:
                    await _process_entry(
                        redis, entry_id, fields, stream, group, DispatchEngine, EventRepository, Ev…
                    )
        except asyncio.CancelledError:
            logger.info("[消费者] 派单消费者收到取消信号，退出")
```

```
            break



        except Exception as exc:   # noqa: BLE001
            if _stream_feature_unsupported(exc):
                logger.warning(
                    "[消费者] 当前 Redis 不支持 Streams 消费组，"
                    "已切换为数据库定时补派；Redis Streams 消费未启用"
                )
                return
            logger.error(f"[消费者] 循环异常：{exc}")
            await asyncio.sleep(3)
async def _process_entry(
    redis,
    entry_id: str,
    fields: dict,
    stream: str,
    group: str,
    DispatchEngine,
    EventRepository,
    EventStatus,
) -> None:
    """处理单条队列消息。"""
    from app.core.exceptions import NoRobotAvailableError
    from app.services.dispatch import finalize_dispatch
    event_id = fields.get("event_id")
    if not event_id:
        await redis.xack(stream, group, entry_id)
        return
    try:
        async with get_session_factory()() as session:
            repo = EventRepository(session)
            event = await repo.get_by_event_id(event_id)
            if event is None or event.status != EventStatus.NEW:
                # 事件不存在或已处理，直接 ACK
                await redis.xack(stream, group, entry_id)
                return
            engine = DispatchEngine(session)
            task = await engine.dispatch_for_event(event)
            await session.commit()
            if task is not None:
                # 下发 + 推送走统一收尾
                await finalize_dispatch(task)
            await redis.xack(stream, group, entry_id)
    except NoRobotAvailableError:
        # 无可用机器人：不 ACK，留在 pending list 等下次补派
        logger.warning(f"[消费者] 事件 {event_id} 暂无可用机器人，稍后重试")
    except Exception as exc:   # noqa: BLE001
        logger.exception(f"[消费者] 处理事件 {event_id} 失败：{exc}")
        # 进死信队列，人工排查
        try:
            await redis.xadd(
```

```
                settings.redis_dead_letter,
                {"event_id": event_id, "error": str(exc)[:500], "entry_id": entry_id},
                maxlen=1000,
                approximate=True,
            )
            await redis.xack(stream, group, entry_id)
        except Exception:   # noqa: BLE001
            pass
async def pending_dispatcher() -> None:
    """定时补派：处理机器人离线期间积压的待派单事件。
    每 30 秒扫描一次，把新事件派给新上线的机器人。
    """
    from app.core.exceptions import NoRobotAvailableError
    from app.models.event import EventStatus
    from app.models.task import TaskStatus
    from app.repositories import EventRepository, TaskRepository
    from app.services.dispatch import DispatchEngine, finalize_dispatch
    # 启动延迟，避免与服务启动竞争
    await asyncio.sleep(10)
    logger.info("[补派] 定时补派任务已启动（间隔 30 秒）")
    while True:
        try:
            session_factory = get_session_factory()
            # 先只快照标量 ID。ORM rollback 会过期对象属性，若在失败后
            # 继续从上一轮的 Task/Event 上读 ID，会抛 MissingGreenlet，
            # 把单条失败升级为整轮循环失败。
            async with session_factory() as session:
                assigned_ids = [
                    task.task_id
                    for task in await TaskRepository(session).list_by_status(
                        TaskStatus.ASSIGNED, limit=50
                    )
                ]
                pending_ids = [
                    task.task_id
                    for task in await TaskRepository(session).list_by_status(
                        TaskStatus.PENDING, limit=50
                    )
                ]
                event_ids = [
                    event.event_id
                    for event in await EventRepository(session).recent_for_dispatch(limit=20)
                ]
            # 第一步：ACK 超时回退（五步筛选的第五步）
            # `handle_ack_timeout` 过去没有任何调用点：机器人掉线后
            #   任务会永久停在 assigned，不报错也不打日志。
            for task_id in assigned_ids:
                try:
                    async with session_factory() as session:
                        task = await TaskRepository(session).get_by_task_id(task_id)
```

```
                    if task is None:
                        continue
                    engine = DispatchEngine(session)
                    if await engine.handle_ack_timeout(task):
                        await session.commit()
            except Exception as exc:   # noqa: BLE001
                logger.warning(f"[补派] 任务 {task_id} 超时回退失败：{exc}")
        # 第二步：换车重派
        # 回退成 pending 的任务若不重新派出去，只是换个状态继续卡着。
        for task_id in pending_ids:
            try:
                async with session_factory() as session:
                    task = await TaskRepository(session).get_by_task_id(task_id)
                    if task is None:
                        continue
                    engine = DispatchEngine(session)
                    if await engine.reassign_task(task):
                        await session.commit()
            except Exception as exc:   # noqa: BLE001
                logger.warning(f"[补派] 任务 {task_id} 换车重派失败：{exc}")
        # 第三步：为尚未派单的事件补派
        dispatched = 0
        for event_id in event_ids:
            try:
                async with session_factory() as session:
                    event = await EventRepository(session).get_by_event_id(event_id)
                    if event is None or event.status != EventStatus.NEW:
                        continue
                    engine = DispatchEngine(session)
                    task = await engine.dispatch_for_event(event)
                    if task is None:
                        continue
                    await session.commit()
                    # 补派也必须下发 + 推送。这里曾经只建任务不通知，
                    #   于是「日志说补派成功」而机器人一动不动。
                    await finalize_dispatch(task)
                    dispatched += 1
            except NoRobotAvailableError:
                continue   # 暂无可用机器人，下一轮再试
            except Exception as exc:   # noqa: BLE001
                logger.warning(f"[补派] 事件 {event_id} 处理失败：{exc}")
                continue
        if dispatched:
            logger.info(f"[补派] 本轮补派 {dispatched} 个任务")
    except asyncio.CancelledError:
        logger.info("[补派] 定时任务收到取消信号，退出")
        break
    except Exception as exc:   # noqa: BLE001
        logger.opt(exception=exc).error(f"[补派] 循环异常：{exc}")
    await asyncio.sleep(30)
```